

C Programming MCQ Question Bank

Hackathon & Competitive Programming Level

165 Comprehensive Questions

14 Sections

With Detailed Explanations

Difficulty Distribution

● Basic ● Medium ● Advanced

Sections Covered

- | | |
|--------------------------|------------------------|
| 1. C Programming Basics | 8. Functions |
| 2. Lexical Elements | 9. Scope Rules |
| 3. Variables & Constants | 10. Files and I/O |
| 4. Data Types | 11. Arrays |
| 5. Operators | 12. Strings |
| 6. Decision Making | 13. Pointers |
| 7. Loops | 14. User-Defined Types |

Tutorials Point India Private Limited
2025

C Programming MCQ Question Bank

Section 1: C Programming Basics

12 Comprehensive Questions

Hackathon & Competitive Programming Level

With Detailed Explanations

Difficulty Levels

● Basic (4) ● Medium (4) ● Advanced (4)

Topics Covered

Compilation Phases • Preprocessing Behavior • Sequence Points
Undefined Behavior • Implementation-Defined Behavior
Memory Layout • Linkage • Storage Duration
Standard Compliance • Platform Dependencies

Section 1: C Programming Basics

Q1. [Basic] What is the output of this program?

```
#include <stdio.h>

int main(void) {
    int x = 10;
    int y = (x++, x++, x++);
    printf("%d %d", x, y);
    return 0;
}
```

- A) 13 12
- B) 13 10
- C) 12 11
- D) 13 11

✓ **Correct Answer: A**

Explanation: The comma operator evaluates left to right and returns the rightmost value. First x++ returns 10 (x becomes 11), second x++ returns 11 (x becomes 12), third x++ returns 12 (x becomes 13). So y = 12, and final x = 13.

Q2. [Basic] Which statement about this code is correct?

```
#include <stdio.h>

int main(void) {
    char *p = "Hello";
    p[0] = 'J';
    printf("%s", p);
    return 0;
}
```

- A) Prints 'Jello'
- B) Prints 'Hello'
- C) Undefined behavior - modifying string literal
- D) Compilation error

✓ **Correct Answer: C**

Explanation: String literals are stored in read-only memory on most systems. Attempting to modify a string literal results in undefined behavior. The program may crash, print 'Jello', or behave unpredictably. Use char p[] = "Hello"; for modifiable strings.

Q3. [Basic] What does this program output?

```
#include <stdio.h>

int main(void) {
    int i = 5;
    printf("%d %d %d", i, i << 2, i >> 1);
    return 0;
}
```

- A) 5 20 2
- B) 5 10 2
- C) 5 20 3
- D) 5 7 4

✓ **Correct Answer: A**

Explanation: $i \ll 2$ shifts bits left by 2 (multiply by 4): $5 * 4 = 20$. $i \gg 1$ shifts bits right by 1 (integer divide by 2): $5 / 2 = 2$. Original i remains 5 as shift operators don't modify the operand.

Q4. [Basic] What happens when this code executes?

```
#include <stdio.h>

int main(void) {
    int arr[3] = {[2] = 30, [0] = 10};
    printf("%d %d %d", arr[0], arr[1], arr[2]);
    return 0;
}
```

- A) 10 0 30
- B) 10 30 0
- C) Compilation error - invalid syntax
- D) 0 10 30

✓ **Correct Answer: A**

Explanation: C99 designated initializers allow specifying indices. $\text{arr}[0] = 10$, $\text{arr}[2] = 30$, and $\text{arr}[1]$ is implicitly initialized to 0. The order in the initializer doesn't matter; indices determine placement.

Q5. [Medium] What is the output of this program?

```
#include <stdio.h>

int f(int *p, int *q) {
    *p = *p + *q;
    *q = *p - *q;
    *p = *p - *q;
    return *p + *q;
}

int main(void) {
    int a = 5, b = 10;
    int result = f(&a, &b);
    printf("%d %d %d", a, b, result);
    return 0;
}
```

- A) 5 10 15
- B) 10 5 15
- C) 10 5 10
- D) 5 10 10

✓ **Correct Answer: B**

Explanation: This swaps a and b without a temp variable. After f(): *p=15, *q=15-10=5, *p=15-5=10. So a=10, b=5. Return value = 10+5=15. This is the arithmetic swap algorithm.

Q6. [Medium] What does this program print?

```
#include <stdio.h>

#define ADD(x, y) x + y
#define MUL(x, y) x * y

int main(void) {
    printf("%d", MUL(ADD(2, 3), ADD(4, 5)));
    return 0;
}
```

- A) 45
- B) 19
- C) 27
- D) 23

✓ **Correct Answer: B**

Explanation: MUL expands to ADD(2,3) * ADD(4,5), then to 2+3 * 4+5. Due to operator precedence (* before +): $2 + (3*4) + 5 = 2 + 12 + 5 = 19$. Proper macros need parentheses: `#define ADD(x,y) ((x)+(y))`.

Q7. [Medium] What is the behavior of this code?

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *p = malloc(sizeof(int) * 5);
    for (int i = 0; i < 5; i++) {
        p[i] = i * 10;
    }
    int *q = p + 2;
    free(p);
    printf("%d", *q);
    return 0;
}
```

- A) 20
- B) 0
- C) Undefined behavior - use after free
- D) Segmentation fault guaranteed

✓ **Correct Answer: C**

Explanation: After `free(p)`, the memory is deallocated. `q` still holds the old address but accessing `*q` is use-after-free, which is undefined behavior. The program might print 20, crash, or produce garbage - behavior is unpredictable.

Q8. [Medium] What does this code output?

```
#include <stdio.h>

int main(void) {
    int x = 0;
    int y = (x = 5) && (x = 10);
    printf("%d %d", x, y);
    return 0;
}
```

- A) 10 1
- B) 5 1
- C) 10 0
- D) 5 0

✓ **Correct Answer: A**

Explanation: `(x = 5)` assigns 5 to `x` and evaluates to 5 (truthy). Since left side of `&&` is true, right side `(x = 10)` is evaluated, assigning 10 to `x`. Both sides are truthy, so `y = 1`. Final: `x=10, y=1`.

Q9. [Advanced] What is the output of this program?

```
#include <stdio.h>

int main(void) {
    struct {
        char a;
        int b;
        char c;
    } s = {'A', 100, 'B'};

    char *p = (char *)&s;
    printf("%c ", *p);
    printf("%c", *(p + sizeof(s) - 1));
    return 0;
}
```

- A) A B
- B) A followed by garbage/undefined
- C) Undefined behavior
- D) A A

✓ **Correct Answer: B**

Explanation: The struct has padding for alignment. 'a' is at offset 0 (prints 'A'). Due to padding, 'c' is likely at offset 5 or 8, not at sizeof(s)-1. The last byte is padding, containing undefined data. Structure layout is implementation-defined.

Q10. [Advanced] What happens when this code executes?

```
#include <stdio.h>
#include <setjmp.h>

jmp_buf buf;
int count = 0;

void func(void) {
    count++;
    if (count < 3) {
        longjmp(buf, count);
    }
}

int main(void) {
    int ret = setjmp(buf);
    printf("%d ", ret);
    if (ret < 3) {
        func();
    }
    return 0;
}
```

- A) 0 1 2

- B) 0 1 2 3
- C) 0 1 2 then infinite loop
- D) Undefined behavior

✓ **Correct Answer: A**

Explanation: setjmp returns 0 initially, prints '0'. func() increments count to 1, longjmp returns to setjmp with value 1, prints '1'. func() increments to 2, longjmp returns 2, prints '2'. func() increments to 3, no longjmp (count not < 3), returns normally, program ends.

Q11. [Advanced] What is the output of this program?

```
#include <stdio.h>

int main(void) {
    int a[][3] = {{1, 2, 3}, {4, 5, 6}};
    int (*p)[3] = a;

    printf("%d ", (*p)[2]);
    p++;
    printf("%d ", (*p)[0]);
    printf("%d", *(*p + 1));
    return 0;
}
```

- A) 3 4 5
- B) 3 4 6
- C) 2 4 5
- D) 3 5 6

✓ **Correct Answer: A**

Explanation: p is pointer to array of 3 ints. Initially p points to a[0]. (*p)[2] = a[0][2] = 3. After p++, p points to a[1]. (*p)[0] = a[1][0] = 4. *(*p + 1) = *(a[1] + 1) = a[1][1] = 5. Output: 3 4 5.

Q12. [Advanced] What is the result of this code?

```
#include <stdio.h>

int x = 5;

int f(void) {
    static int x = 1;
    return x++;
}

int g(void) {
    return x++;
}

int main(void) {
    printf("%d ", f() + g());
    printf("%d ", f() + g());
    printf("%d", x);
    return 0;
}
```

- A) 6 8 7
- B) 6 9 7
- C) 6 8 5
- D) 5 7 7

✓ **Correct Answer: A**

Explanation: First call: f() returns 1 (static x becomes 2), g() returns 5 (global x becomes 6). Sum = 6. Second call: f() returns 2 (static x becomes 3), g() returns 6 (global x becomes 7). Sum = 8. Final global x = 7.

— End of Section 1: C Programming Basics —

12 Questions | Part of 165-Question Bank
© 2026 Tutorials Point India Private Limited

C Programming MCQ Question Bank

Section 2: Lexical Elements

12 Comprehensive Questions

Hackathon & Competitive Programming Level

With Detailed Explanations

Difficulty Levels

● Basic (4) ● Medium (4) ● Advanced (4)

Topics Covered

Tokenization • Maximal Munch Rule • Escape Sequences
Trigraphs & Digraphs • Identifier Rules • Keywords
String Literal Concatenation • Character Constants
Preprocessing Tokens • Operator Precedence

Section 2: Lexical Elements

Q1. [Basic] How many tokens are in this statement?

```
int x = a++ + ++b;
```

- A) 7
- B) 8
- C) 9
- D) 10

✓ **Correct Answer: C**

Explanation: Tokens: 'int' (keyword), 'x' (identifier), '=' (operator), 'a' (identifier), '++' (operator), '+' (operator), '++' (operator), 'b' (identifier), ';' (punctuator) = 9 tokens. The lexer uses maximal munch to parse a++ and ++b correctly.

Q2. [Basic] What is the output of this program?

```
#include <stdio.h>

int main(void) {
    printf("Hello" "World" "!");
    return 0;
}
```

- A) HelloWorld!
- B) Hello World !
- C) Compilation error
- D) HelloWorld !

✓ **Correct Answer: A**

Explanation: Adjacent string literals are concatenated during translation phase 6 (before compilation). "Hello" "World" "!" becomes "HelloWorld!". No spaces are added between them.

Q3. [Basic] What does this code print?

```
#include <stdio.h>

int main(void) {
    int x = 10;
    int y = 0x1A + 012 + 5;
    printf("%d", y);
    return 0;
}
```

- A) 27
- B) 41
- C) 35
- D) 31

✓ **Correct Answer: C**

Explanation: 0x1A is hexadecimal = 26 decimal. 012 is octal (leading zero) = 10 decimal. 5 is decimal. Total: $26 + 10 + 5 = 35$. Octal literals start with 0, hex with 0x.

Q4. [Basic] What is the value of ch?

```
#include <stdio.h>

int main(void) {
    char ch = '\101';
    printf("%c %d", ch, ch);
    return 0;
}
```

- A) A 65
- B) 1 49
- C) \ 92
- D) Compilation error

✓ **Correct Answer: A**

Explanation: '\101' is an octal escape sequence. Octal 101 = $1*64 + 0*8 + 1 = 65$ in decimal, which is ASCII 'A'. Octal escapes use 1-3 digits after backslash.

Q5. [Medium] What is the output?

```
#include <stdio.h>

int main(void) {
    int a = 0xABCDEF;
    int b = a & 0xFF00FF;
    printf("%X", b);
    return 0;
}
```

- A) ABCDEF
- B) AB00EF
- C) ABEF
- D) CD

✓ **Correct Answer: B**

Explanation: 0xABCDEF & 0xFF00FF performs bitwise AND. For each hex digit: $AB \& FF = AB$, $CD \& 00 = 00$, $EF \& FF = EF$. Result: 0xAB00EF. The mask 0xFF00FF extracts bytes 0 and 2, zeroing byte 1.

Q6. [Medium] How is the expression parsed?

```
int x = a---b;
```

- A) a - (--b)
- B) (a--) - b
- C) a - (-(-b))
- D) Compilation error

✓ **Correct Answer: B**

Explanation: Maximal munch rule: lexer takes longest valid token. 'a--' is valid (post-decrement), leaving '-b'. So it parses as (a--) - b. The expression a---b has 5 tokens: a, --, -, b, ;

Q7. [Medium] What does this code output?

```
#include <stdio.h>

int main(void) {
    char *s = "ab\
cd\
ef";
    printf("%zu", strlen(s));
    return 0;
}
```

- A) 6
- B) 10
- C) 8
- D) 12

✓ **Correct Answer: A**

Explanation: Backslash-newline (line splicing) joins physical lines during translation phase 2. The string becomes "abcdef" with length 6. The backslash-newline sequences are removed entirely.

Q8. [Medium] What is the output?

```
#include <stdio.h>

int main(void) {
    int val = '\x41' + '\x42' + '\x43';
    printf("%d", val);
    return 0;
}
```

- A) 198
- B) 126
- C) ABC
- D) 656667

✓ **Correct Answer: A**

Explanation: '\x41' is hex escape for 65 ('A'), '\x42' is 66 ('B'), '\x43' is 67 ('C'). In C, character constants have type int. Sum: 65 + 66 + 67 = 198.

Q9. [Advanced] What is the result of this code?

```
#include <stdio.h>

#define STR(x) #x
#define CONCAT(a, b) a ## b

int main(void) {
    int xy = 100;
    printf("%s ", STR(CONCAT(x, y)));
    printf("%d", CONCAT(x, y));
    return 0;
}
```

- A) CONCAT(x, y) 100
- B) xy 100
- C) CONCAT(x, y) xy
- D) x y 100

✓ **Correct Answer: A**

Explanation: # (stringification) happens BEFORE macro expansion of its argument. STR(CONCAT(x,y)) stringifies 'CONCAT(x,y)' literally. But CONCAT(x,y) in printf expands to xy = 100. Stringification prevents expansion.

Q10. [Advanced] What is the behavior of this code?

```
#include <stdio.h>

int main(void) {
    int arr[3] = {1, 2, 3};
    int i = 0;
    int val = arr[i+++1];
    printf("%d %d", val, i);
    return 0;
}
```

- A) 2 1
- B) 1 1

C) 3 1

D) 2 2

✓ **Correct Answer: A**

Explanation: `i+++1` is parsed as `(i++) + 1` by maximal munch. `i++` returns 0 (then `i` becomes 1), so `index = 0 + 1 = 1`. `arr[1] = 2`. Final: `val = 2`, `i = 1`.

Q11. [Advanced] What does this program output?

```
#include <stdio.h>

int main(void) {
    char *p = "\
Hello";
    char *q = "\\Hello";
    printf("%d %d", (int)strlen(p), (int)strlen(q));
    return 0;
}
```

A) 5 6

B) 6 6

C) 5 5

D) 6 5

✓ **Correct Answer: A**

Explanation: In `p`, backslash-newline is line splicing (removed), giving "Hello" (length 5). In `q`, `'\\H'` is not a valid escape sequence. C allows unknown escapes as implementation-defined; typically just 'H', giving "Hello" (length 6 with the backslash or 5 without). Most compilers treat `\\H` as H, so length is 6 if backslash kept.

Q12. [Advanced] What is the output?

```
#include <stdio.h>

#define XSTR(x) STR(x)
#define STR(x) #x
#define FOO 42

int main(void) {
    printf("%s %s", STR(FOO), XSTR(FOO));
    return 0;
}
```

A) FOO FOO

B) 42 42

C) FOO 42

D) 42 FOO

✓ **Correct Answer: C**

Explanation: STR(FOO) stringifies immediately: "FOO". XSTR(FOO) first expands FOO to 42, then calls STR(42), giving "42". This two-level macro trick is needed to stringify macro values. Direct stringification prevents expansion.

— End of Section 2: Lexical Elements —

12 Questions | Part of 165-Question Bank
© 2026 Tutorials Point India Private Limited

C Programming MCQ Question Bank

Section 3: Variables & Constants

12 Comprehensive Questions

Hackathon & Competitive Programming Level

With Detailed Explanations

Difficulty Levels

● Basic (4) ● Medium (4) ● Advanced (4)

Topics Covered

Storage Classes • Linkage • Storage Duration
const vs #define • volatile Qualifier • Tentative Definitions
extern Declarations • Static Variables • Register Keyword
Initialization Rules • Compound Literals • Enumerations

Section 3: Variables & Constants

Q1. [Basic] What is the output of this program?

```
#include <stdio.h>

int x;
int x = 10;
int x;

int main(void) {
    printf("%d", x);
    return 0;
}
```

- A) Compilation error - redefinition
- B) 10
- C) 0
- D) Undefined behavior

✓ **Correct Answer: B**

Explanation: In C, 'int x;' at file scope without initializer is a tentative definition. Multiple tentative definitions are allowed and merge into one. 'int x = 10;' provides the actual definition. All refer to the same object. Output: 10.

Q2. [Basic] What does this code print?

```
#include <stdio.h>

void func(void) {
    static int count = 0;
    count += 5;
    printf("%d ", count);
}

int main(void) {
    func();
    func();
    func();
    return 0;
}
```

- A) 5 5 5
- B) 5 10 15
- C) 0 5 10
- D) 5 5 5 5

✓ **Correct Answer: B**

Explanation: Static local variables retain their value between function calls and are initialized only once. First call: 0+5=5. Second call: 5+5=10. Third call: 10+5=15. Output: 5 10 15.

Q3. [Basic] What is the output?

```
#include <stdio.h>

int main(void) {
    enum color { RED = 5, GREEN, BLUE = 10, YELLOW };
    printf("%d %d %d %d", RED, GREEN, BLUE, YELLOW);
    return 0;
}
```

- A) 5 6 10 11
- B) 0 1 2 3
- C) 5 5 10 10
- D) 5 10 10 11

✓ Correct Answer: A

Explanation: Enum values: RED=5 (explicit), GREEN=6 (RED+1), BLUE=10 (explicit), YELLOW=11 (BLUE+1). Unassigned enum constants take the value of previous constant plus one.

Q4. [Basic] What is the behavior of this code?

```
#include <stdio.h>

int main(void) {
    const int size = 10;
    int arr[size];
    printf("%zu", sizeof(arr));
    return 0;
}
```

- A) 40 (assuming 4-byte int)
- B) Compilation error in C89
- C) 10
- D) Runtime error

✓ Correct Answer: A

Explanation: In C99+, this creates a VLA (Variable Length Array) since 'const int' is not a compile-time constant in C. The array has 10 elements, so sizeof = 10 * sizeof(int) = 40. In C89 this would be an error.

Q5. [Medium] What is the output of this program?

```

#include <stdio.h>

int a = 1;

void foo(void) {
    int a = 2;
    {
        extern int a;
        printf("%d ", a);
    }
    printf("%d", a);
}

int main(void) {
    foo();
    return 0;
}

```

- A) 2 2
- B) 1 2
- C) 2 1
- D) 1 1

✓ **Correct Answer: B**

Explanation: Inside the inner block, 'extern int a' refers to the global a (value 1), bypassing the local a=2. After the block ends, the local a=2 is visible again. Output: 1 2.

Q6. [Medium] What does this code print?

```

#include <stdio.h>

int main(void) {
    int i = 3;
    int *p = (int[]){10, 20, 30, 40};
    p += i;
    printf("%d %d", *p, *(p - 2));
    return 0;
}

```

- A) 40 20
- B) 30 10
- C) 40 10
- D) Undefined behavior

✓ **Correct Answer: A**

Explanation: Compound literal creates an array {10,20,30,40}. p initially points to element 0. p += 3 makes p point to element 3 (value 40). p-2 points to element 1 (value 20). Output: 40 20.

Q7. [Medium] What is the result of this code?

```
#include <stdio.h>

static int x = 5;
static int y;

int main(void) {
    static int z;
    printf("%d %d %d", x, y, z);
    return 0;
}
```

- A) 5 0 0
- B) 5 garbage garbage
- C) 5 0 garbage
- D) Compilation error

✓ Correct Answer: A

Explanation: Static variables (both file-scope and block-scope) are zero-initialized if not explicitly initialized. x=5 (explicit), y=0 (implicit), z=0 (implicit). All static storage duration variables default to zero.

Q8. [Medium] What is the output?

```
#include <stdio.h>

int main(void) {
    int x = 5;
    int *const p = &x;
    int y = 10;
    *p = 20;
    // p = &y; // Would this compile?
    printf("%d", x);
    return 0;
}
```

- A) 5, and p = &y; would compile
- B) 20, and p = &y; would NOT compile
- C) 20, and p = &y; would compile
- D) Compilation error

✓ Correct Answer: B

Explanation: 'int *const p' is a const pointer to int. The pointer itself cannot be changed (p = &y; fails), but the pointed-to value can be modified (*p = 20 works). x becomes 20.

Q9. [Advanced] What is the behavior of this code?

```

#include <stdio.h>

volatile int flag = 0;

void handler(void) {
    flag = 1;
}

int main(void) {
    while (!flag) {
        // busy wait
    }
    printf("Done");
    return 0;
}

```

- A) Always prints 'Done' eventually
- B) Without volatile, compiler might optimize to infinite loop
- C) volatile has no effect here
- D) Compilation error

✓ **Correct Answer: B**

Explanation: Without volatile, the compiler may cache 'flag' in a register and never re-read it from memory, creating an infinite loop. volatile forces the compiler to read from memory each time, essential for signal handlers, hardware registers, and multi-threaded code.

Q10. [Advanced] What does this code output?

```

#include <stdio.h>

int main(void) {
    int x = 10;
    {
        int x = x;
        printf("%d ", x);
    }
    printf("%d", x);
    return 0;
}

```

- A) 10 10
- B) 0 10
- C) Undefined value, then 10
- D) Compilation error

✓ **Correct Answer: C**

Explanation: In 'int x = x;', the inner x shadows the outer x BEFORE the initializer is evaluated. So inner x is initialized with its own uninitialized value (undefined). The outer x remains 10. This is a common trap.

Q11. [Advanced] What is the result?

```
#include <stdio.h>

int foo(void) {
    static int x = 0;
    return ++x;
}

int main(void) {
    int a = foo() + foo() + foo();
    printf("%d", a);
    return 0;
}
```

- A) 6
- B) 3
- C) Unspecified value (1+2+3 in some order)
- D) 9

✓ Correct Answer: A

Explanation: While evaluation order of + operands is unspecified, foo() will return 1, 2, 3 in some order. The sum 1+2+3=6 is always the same regardless of order. Static x increments across all calls.

Q12. [Advanced] What is the output of this program?

```
#include <stdio.h>

int main(void) {
    const int a = 10;
    enum { b = 10 };

    switch(5) {
        // case a: printf("a"); break; // Line X
        case b: printf("b"); break;
        default: printf("default");
    }
    return 0;
}
```

- A) Prints 'b', and Line X would also compile
- B) Prints 'b', but Line X would NOT compile
- C) Prints 'default'
- D) Compilation error

✓ Correct Answer: B

Explanation: case labels require integer constant expressions. 'enum { b = 10 }' creates a true compile-time constant. But 'const int a' is a read-only variable in C, not a constant expression. Line X would fail: 'case a' is

illegal in C.

— End of Section 3: Variables & Constants —

12 Questions | Part of 165-Question Bank
© 2026 Tutorials Point India Private Limited

C Programming MCQ Question Bank

Section 4: Data Types

12 Comprehensive Questions

Hackathon & Competitive Programming Level

With Detailed Explanations

Difficulty Levels

● Basic (4) ● Medium (4) ● Advanced (4)

Topics Covered

Integer Promotion • Usual Arithmetic Conversions
Signed vs Unsigned • Overflow Behavior • Type Limits
Floating Point Precision • IEEE 754 Quirks
sizeof Operator • Type Punning • Implicit Conversions

Section 4: Data Types

Q1. [Basic] What is the output of this program?

```
#include <stdio.h>

int main(void) {
    unsigned int a = 1;
    int b = -1;
    if (a > b)
        printf("a > b");
    else
        printf("a <= b");
    return 0;
}
```

- A) a > b
- B) a <= b
- C) Compilation error
- D) Undefined behavior

✓ **Correct Answer: B**

Explanation: When comparing signed and unsigned of same rank, signed is converted to unsigned. -1 becomes `UINT_MAX` (a very large number). So `1 > UINT_MAX` is false. Output: 'a <= b'. This is a classic signed/unsigned comparison trap.

Q2. [Basic] What does this code print?

```
#include <stdio.h>

int main(void) {
    char c = 255;
    printf("%d", c);
    return 0;
}
```

- A) 255
- B) -1
- C) Implementation-defined
- D) 0

✓ **Correct Answer: C**

Explanation: Whether 'char' is signed or unsigned is implementation-defined. If signed (common), 255 overflows to -1 (two's complement). If unsigned, it stays 255. The result depends on the platform/compiler.

Q3. [Basic] What is the output?

```
#include <stdio.h>

int main(void) {
    int x = 5;
    double y = 2;
    printf("%f", x / y + x % (int)y);
    return 0;
}
```

- A) 3.500000
- B) 2.500000
- C) 3.000000
- D) 2.000000

✓ **Correct Answer: A**

Explanation: x/y : int 5 converts to double, $5.0/2.0 = 2.5$. $x\%(int)y$: $5\%2 = 1$. Total: $2.5 + 1 = 3.5$. Note: % requires integer operands, so cast is needed.

Q4. [Basic] What is the result?

```
#include <stdio.h>

int main(void) {
    unsigned char a = 200;
    unsigned char b = 100;
    unsigned char c = a + b;
    printf("%d", c);
    return 0;
}
```

- A) 300
- B) 44
- C) -212
- D) Undefined behavior

✓ **Correct Answer: B**

Explanation: $a + b = 300$, but unsigned char can only hold 0-255. $300 \% 256 = 44$. Unsigned overflow wraps around (well-defined modulo arithmetic). No undefined behavior for unsigned types.

Q5. [Medium] What does this code output?

```
#include <stdio.h>

int main(void) {
    float f = 0.1f;
    double d = 0.1;
    if (f == d)
        printf("Equal");
    else if (f > d)
        printf("Float > Double");
    else
        printf("Float < Double");
    return 0;
}
```

- A) Equal
- B) Float > Double
- C) Float < Double
- D) Implementation-defined

✓ **Correct Answer: B**

Explanation: 0.1 cannot be exactly represented in binary. Float has less precision than double. When f is promoted to double for comparison, its rounded value (0.100000001...) is slightly larger than double's more precise representation (0.10000000000000000555...).

Q6. [Medium] What is the output?

```
#include <stdio.h>
#include <limits.h>

int main(void) {
    int x = INT_MAX;
    unsigned int y = (unsigned int)x + 1;
    printf("%u", y);
    return 0;
}
```

- A) 0
- B) 2147483648 (assuming 32-bit int)
- C) Undefined behavior
- D) -2147483648

✓ **Correct Answer: B**

Explanation: INT_MAX is 2147483647 (for 32-bit). Cast to unsigned preserves bit pattern. Adding 1 to unsigned is well-defined: 2147483647 + 1 = 2147483648. No overflow issue because unsigned arithmetic wraps.

Q7. [Medium] What is the result of this code?

```
#include <stdio.h>

int main(void) {
    short a = 32767;
    short b = 1;
    int c = a + b;
    printf("%d", c);
    return 0;
}
```

- A) -32768
- B) 32768
- C) Undefined behavior
- D) 0

✓ **Correct Answer: B**

Explanation: Integer promotion: short values are promoted to int BEFORE arithmetic. So a and b become ints (32767 and 1), their sum is 32768, stored in int c. No overflow because int can hold this value.

Q8. [Medium] What does this print?

```
#include <stdio.h>

int main(void) {
    long long ll = 1LL << 40;
    int i = ll;
    printf("%d", i);
    return 0;
}
```

- A) 1099511627776
- B) 0
- C) Implementation-defined (likely 0)
- D) Compilation error

✓ **Correct Answer: C**

Explanation: $1LL \ll 40$ is a 64-bit value. Assigning to 32-bit int causes truncation. The lower 32 bits of 2^{40} are all zeros, so i is likely 0. But conversion of out-of-range values to signed types is implementation-defined.

Q9. [Advanced] What is the output?

```
#include <stdio.h>

int main(void) {
    double d = 1e308;
    d = d * 10.0;
    if (d == d)
        printf("Equal");
    else
        printf("Not Equal");
    return 0;
}
```

- A) Equal
- B) Not Equal
- C) Undefined behavior
- D) Runtime error

✓ **Correct Answer: A**

Explanation: $1e308 * 10$ overflows to infinity (IEEE 754). $\text{Infinity} == \text{Infinity}$ is true in IEEE 754. Only NaN is not equal to itself. So d (which is +Inf) equals itself. Output: 'Equal'.

Q10. [Advanced] What does this code print?

```
#include <stdio.h>

int main(void) {
    int x = -5;
    unsigned int y = 3;
    printf("%d", (x / y) > 0);
    return 0;
}
```

- A) 0
- B) 1
- C) -1
- D) Undefined behavior

✓ **Correct Answer: B**

Explanation: In x/y , signed x converts to unsigned. -5 becomes a huge unsigned value ($\text{UINT_MAX} - 4$). Dividing by 3 gives a large positive result. This is definitely > 0 , so output is 1 (true).

Q11. [Advanced] What is the result?

```
#include <stdio.h>
#include <stdint.h>

int main(void) {
    uint32_t a = 0x80000000;
    int32_t b = (int32_t)a;
    int64_t c = (int64_t)a;
    int64_t d = (int64_t)b;
    printf("%lld %lld", (long long)c, (long long)d);
    return 0;
}
```

- A) 2147483648 2147483648
- B) 2147483648 -2147483648
- C) -2147483648 -2147483648
- D) Undefined behavior

✓ **Correct Answer: B**

Explanation: a = 0x80000000 (unsigned). c = (int64_t)a extends unsigned to 64-bit: 2147483648. b = (int32_t)a interprets as signed: -2147483648. d = (int64_t)b sign-extends: -2147483648. Different conversion paths give different results!

Q12. [Advanced] What is the behavior of this code?

```
#include <stdio.h>
#include <math.h>

int main(void) {
    double d = 0.0 / 0.0;
    if (d != d)
        printf("NaN detected");
    else
        printf("Not NaN");
    return 0;
}
```

- A) NaN detected
- B) Not NaN
- C) Undefined behavior
- D) Division by zero error

✓ **Correct Answer: A**

Explanation: 0.0/0.0 produces NaN (Not a Number) in IEEE 754 floating point. The unique property of NaN is that it's not equal to itself (NaN != NaN is true). This is the standard way to detect NaN: (x != x).

C Programming MCQ Question Bank

Section 5: Operators

12 Comprehensive Questions

Hackathon & Competitive Programming Level

With Detailed Explanations

Difficulty Levels

● Basic (4) ● Medium (4) ● Advanced (4)

Topics Covered

Operator Precedence & Associativity • Bitwise Tricks
Short-Circuit Evaluation • Ternary Operator Nuances
Shift Operators • Modulo with Negatives
Assignment Expressions • Logical vs Bitwise • sizeof Quirks

Section 5: Operators

Q1. [Basic] What is the output of this program?

```
#include <stdio.h>

int main(void) {
    int a = 5, b = 3;
    int c = a > b ? a++, b++ : a--, b--;
    printf("%d %d %d", a, b, c);
    return 0;
}
```

- A) 6 4 3
- B) 6 4 5
- C) 5 3 3
- D) 6 3 3

✓ **Correct Answer: A**

Explanation: $a > b$ is true, so $(a++, b++)$ executes. Comma operator: $a++$ returns 5, a becomes 6, then $b++$ returns 3, b becomes 4. The comma expression returns rightmost value (3). So $c = 3$, $a = 6$, $b = 4$.

Q2. [Basic] What does this code print?

```
#include <stdio.h>

int main(void) {
    int x = -7 % 3;
    int y = 7 % -3;
    printf("%d %d", x, y);
    return 0;
}
```

- A) -1 1
- B) 2 -2
- C) -1 -1
- D) 1 1

✓ **Correct Answer: A**

Explanation: In C99+, the result of $\%$ has the same sign as the dividend. $-7 \% 3$: dividend is -7 (negative), so result is -1. $7 \% -3$: dividend is 7 (positive), so result is 1. Formula: $a \% b$ has sign of a .

Q3. [Basic] What is the output?

```
#include <stdio.h>

int main(void) {
    int a = 10, b = 20, c = 30;
    int result = a < b < c;
    printf("%d", result);
    return 0;
}
```

- A) 1
- B) 0
- C) 30
- D) Compilation error

✓ **Correct Answer: A**

Explanation: This is NOT mathematical chaining! Parsed left-to-right: $(a < b) < c$. $(10 < 20) = 1$ (true). Then $1 < 30 = 1$ (true). Result is 1. To properly chain: $(a < b) \ \&\& \ (b < c)$.

Q4. [Basic] What is the result?

```
#include <stdio.h>

int main(void) {
    int x = 5;
    int y = x & (x - 1);
    printf("%d", y);
    return 0;
}
```

- A) 4
- B) 5
- C) 1
- D) 0

✓ **Correct Answer: A**

Explanation: $x \ \& \ (x-1)$ clears the lowest set bit. $5 = 0101$, $4 = 0100$. $0101 \ \& \ 0100 = 0100 = 4$. This trick is used to check if a number is a power of 2 (result is 0 if power of 2).

Q5. [Medium] What does this code output?

```
#include <stdio.h>

int main(void) {
    int a = 1, b = 2, c = 3;
    int x = a | b & c;
    int y = (a | b) & c;
    printf("%d %d", x, y);
    return 0;
}
```

- A) 3 3
- B) 3 1
- C) 1 3
- D) 1 1

✓ **Correct Answer: A**

Explanation: & has higher precedence than |. $x = a | (b \& c) = 1 | (2 \& 3) = 1 | 2 = 3$. $y = (a | b) \& c = (1 | 2) \& 3 = 3 \& 3 = 3$. Both are 3, but for different reasons!

Q6. [Medium] What is the output?

```
#include <stdio.h>

int f(int *x) {
    return (*x)++;
}

int main(void) {
    int a = 5;
    int b = f(&a) + f(&a);
    printf("%d %d", a, b);
    return 0;
}
```

- A) 7 11
- B) 7 10
- C) 7, unspecified value for b
- D) 6 10

✓ **Correct Answer: C**

Explanation: Each $f(\&a;)$ modifies a and returns old value. But the order of evaluating $f(\&a;) + f(\&a;)$ is unspecified. Could be $5+6=11$ or $6+5=11$, but a will be 7. The sum is same but evaluation order is unspecified by standard.

Q7. [Medium] What does this print?

```
#include <stdio.h>

int main(void) {
    int i = 8;
    int j = i >> 1 >> 1 >> 1;
    int k = i >> (1 + 1 + 1);
    printf("%d %d", j, k);
    return 0;
}
```

- A) 1 1
- B) 1 8
- C) 8 1
- D) 2 1

✓ **Correct Answer: A**

Explanation: >> is left-to-right associative. $j = ((i \gg 1) \gg 1) \gg 1 = ((8 \gg 1) \gg 1) \gg 1 = (4 \gg 1) \gg 1 = 2 \gg 1 = 1$. $k = 8 \gg 3 = 1$. Both equal 1.

Q8. [Medium] What is the result?

```
#include <stdio.h>

int main(void) {
    int x = 3;
    int y = 2;
    int z = x ^ y ^ x ^ y ^ x;
    printf("%d", z);
    return 0;
}
```

- A) 0
- B) 3
- C) 2
- D) 1

✓ **Correct Answer: B**

Explanation: XOR properties: $a \wedge a = 0$, $a \wedge 0 = a$, XOR is associative and commutative. $x \wedge y \wedge x \wedge y \wedge x = (x \wedge x) \wedge (y \wedge y) \wedge x = 0 \wedge 0 \wedge x = x = 3$. Pairs cancel out, one x remains.

Q9. [Advanced] What is the output?

```
#include <stdio.h>

int main(void) {
    int a = 5, b = 5;
    int x = a == b ? a++ : b--;
    int y = a != b ? ++a : --b;
    printf("%d %d %d %d", a, b, x, y);
    return 0;
}
```

- A) 7 5 5 7
- B) 6 5 5 6
- C) 7 4 5 7
- D) 6 4 5 6

✓ **Correct Answer: A**

Explanation: `a==b` is true (`5==5`), so `a++` executes: `x=5` (post-inc returns 5), `a` becomes 6. Now `a!=b` is true (`6!=5`), so `++a` executes: `a` becomes 7, `y=7` (pre-inc returns new value). Final: `a=7`, `b=5`, `x=5`, `y=7`.

Q10. [Advanced] What does this code output?

```
#include <stdio.h>

int main(void) {
    unsigned int x = 1;
    int shift = -1;
    printf("%u", x << shift);
    return 0;
}
```

- A) 0
- B) 2147483648
- C) Undefined behavior
- D) 1

✓ **Correct Answer: C**

Explanation: Shifting by a negative amount is undefined behavior in C. The standard says if the shift count is negative or `>=` width of type, behavior is undefined. Compilers may produce any result.

Q11. [Advanced] What is the result?

```
#include <stdio.h>

int main(void) {
    int arr[] = {10, 20, 30, 40};
    int *p = arr;
    int x = *p++;
    int y = ++*p;
    int z = *++p;
    printf("%d %d %d", x, y, z);
    return 0;
}
```

- A) 10 21 30
- B) 10 20 30
- C) 20 21 30
- D) 10 21 40

✓ **Correct Answer: A**

Explanation: *p++ = *(p++): returns *p (10), then p moves to arr[1]. ++*p = ++(*p): increments arr[1] from 20 to 21, returns 21. *++p = *(++p): p moves to arr[2], returns 30. Result: 10 21 30.

Q12. [Advanced] What is the output?

```
#include <stdio.h>

int main(void) {
    int x = 2, y = 0;
    int a = (x *= 3, y += x, x + y);
    int b = x == 6 && y == 6 || y == 0;
    printf("%d %d", a, b);
    return 0;
}
```

- A) 12 1
- B) 6 1
- C) 12 0
- D) 8 1

✓ **Correct Answer: A**

Explanation: Comma expression: x *= 3 (x=6), y += x (y=6), returns x+y = 12. a=12. For b: (x==6 && y==6) || y==0 = (true && true) || false = true || false = 1. So a=12, b=1.

— End of Section 5: Operators —

C Programming MCQ Question Bank

Section 6: Decision Making

12 Comprehensive Questions

Hackathon & Competitive Programming Level

With Detailed Explanations

Difficulty Levels

● Basic (4) ● Medium (4) ● Advanced (4)

Topics Covered

Dangling Else Problem • Switch Fall-through
Duff's Device • Nested Conditionals • goto Statement
Short-Circuit in Conditions • Switch with Ranges
Conditional Compilation • Complex Boolean Logic

Section 6: Decision Making

Q1. [Basic] What is the output of this program?

```
#include <stdio.h>

int main(void) {
    int x = 2;
    switch (x) {
        case 1:
        case 2:
        case 3:
            printf("A");
        case 4:
            printf("B");
            break;
        default:
            printf("C");
    }
    return 0;
}
```

- A) A
- B) AB
- C) ABC
- D) B

✓ **Correct Answer: B**

Explanation: x=2 matches case 2. Cases 1,2,3 have no statements before case 4, so execution falls through. Prints 'A', then falls to case 4, prints 'B', then break exits. Output: AB.

Q2. [Basic] What does this code print?

```
#include <stdio.h>

int main(void) {
    int a = 0, b = 0, c = 0;
    if (a)
        if (b)
            c = 1;
    else
        c = 2;
    printf("%d", c);
    return 0;
}
```

- A) 0
- B) 1

- C) 2
- D) Compilation error

✓ **Correct Answer: A**

Explanation: Dangling else: 'else' binds to nearest unmatched 'if', which is 'if (b)'. Since a=0 is false, the outer if body (including nested if-else) is skipped entirely. c remains 0.

Q3. [Basic] What is the output?

```
#include <stdio.h>

int main(void) {
    int x = 5;
    if (x = 10)
        printf("True: %d", x);
    else
        printf("False: %d", x);
    return 0;
}
```

- A) False: 5
- B) True: 10
- C) False: 10
- D) Compilation error

✓ **Correct Answer: B**

Explanation: x = 10 is assignment, not comparison (==). It assigns 10 to x and returns 10. Since 10 is non-zero (truthy), the if-branch executes. Output: True: 10. Classic bug!

Q4. [Basic] What is the result?

```
#include <stdio.h>

int main(void) {
    int n = 5;
    switch (n) {
        default:
            printf("D");
        case 1:
            printf("1");
            break;
        case 5:
            printf("5");
    }
    return 0;
}
```

- A) D1

- B) 5
- C) D15
- D) D

✓ **Correct Answer: B**

Explanation: default doesn't have to be last! n=5 matches case 5 directly, prints '5'. No break after case 5, but switch ends anyway. default is only executed if no case matches. Output: 5.

Q5. [Medium] What does this code output?

```
#include <stdio.h>

int check(int x) {
    printf("%d ", x);
    return x;
}

int main(void) {
    if (check(0) || check(1) || check(2))
        printf("True");
    else
        printf("False");
    return 0;
}
```

- A) 0 1 2 True
- B) 0 1 True
- C) 0 False
- D) 0 1 2 False

✓ **Correct Answer: B**

Explanation: Short-circuit evaluation: || stops at first true. check(0) returns 0 (false), continue. check(1) returns 1 (true), stop evaluating. check(2) never called. Output: 0 1 True.

Q6. [Medium] What is the output?

```
#include <stdio.h>

int main(void) {
    int i = 0;
    switch (i) {
        case 0:
            i++;
        case 1:
            i += 2;
        case 2:
            i += 3;
        default:
            i += 4;
    }
    printf("%d", i);
    return 0;
}
```

- A) 1
- B) 3
- C) 6
- D) 10

✓ **Correct Answer: D**

Explanation: No break statements! i=0 matches case 0. i++ makes i=1. Falls to case 1: i+=2 makes i=3. Falls to case 2: i+=3 makes i=6. Falls to default: i+=4 makes i=10. Total fall-through.

Q7. [Medium] What does this print?

```
#include <stdio.h>

int main(void) {
    int x = 1, y = 2, z = 3;
    int result = x < y ? y < z ? z : y : x;
    printf("%d", result);
    return 0;
}
```

- A) 1
- B) 2
- C) 3
- D) 0

✓ **Correct Answer: C**

Explanation: Nested ternary: $x < y ? (y < z ? z : y) : x$. $1 < 2$ is true, evaluate inner: $2 < 3$ is true, return $z = 3$. Ternary associates right-to-left. Result: 3.

Q8. [Medium] What is the output?

```
#include <stdio.h>

int main(void) {
    int a = 1;
    switch (a) {
        int b = 10;
        case 1:
            printf("%d", b);
            break;
    }
    return 0;
}
```

- A) 10
- B) Garbage/Undefined value
- C) 0
- D) Compilation error

✓ Correct Answer: B

Explanation: Declaration 'int b = 10;' is valid inside switch but initialization is skipped when jumping to case 1. b exists (declaration is processed) but is uninitialized. Accessing it gives undefined value. C99+ may error if b is VLA.

Q9. [Advanced] What is the output of this code?

```
#include <stdio.h>

int main(void) {
    int x = 0;
    if (x++) {
        if (x++ > 1)
            printf("A");
        else
            printf("B");
    } else {
        if (++x > 1)
            printf("C");
        else
            printf("D");
    }
    printf("%d", x);
    return 0;
}
```

- A) C2
- B) D2
- C) A2
- D) B2

✓ **Correct Answer: A**

Explanation: if(x++): x=0 (false), then x becomes 1. Takes else branch. ++x makes x=2, returns 2. 2 > 1 is true, prints 'C'. Final x=2. Output: C2.

Q10. [Advanced] What does this code output?

```
#include <stdio.h>

int main(void) {
    int i = 3;
    switch (i) {
        case 1: printf("1"); break;
        case 2: printf("2"); break;
        case 1+2: printf("3"); break;
        case 2*2: printf("4"); break;
        default: printf("D");
    }
    return 0;
}
```

- A) 3
- B) 34
- C) Compilation error - duplicate case
- D) D

✓ **Correct Answer: A**

Explanation: Case labels can be constant expressions. 1+2=3, 2*2=4. All unique, no duplicate. i=3 matches case 1+2 (which is case 3). Prints '3' and breaks. Output: 3.

Q11. [Advanced] What is the result?

```

#include <stdio.h>

int main(void) {
    int n = 2;
    int result = 0;
    switch (n) {
        case 2:
            result = 1;
            goto skip;
        case 3:
            result = 2;
        skip:
            result += 10;
            break;
        default:
            result = -1;
    }
    printf("%d", result);
    return 0;
}

```

- A) 1
- B) 11
- C) 12
- D) Compilation error

✓ **Correct Answer: B**

Explanation: n=2 matches case 2. result=1. goto skip jumps to label 'skip' (inside case 3). result += 10 makes result=11. break exits switch. Output: 11. goto can jump within switch!

Q12. [Advanced] What does this code print?

```

#include <stdio.h>

int f(int *p) {
    (*p)++;
    return 1;
}

int main(void) {
    int x = 0;
    if (f(&x) && f(&x) && f(&x)) {
        printf("All true: ");
    }
    printf("%d", x);
    return 0;
}

```

- A) All true: 1
- B) All true: 3

C) 1

D) 3

✓ **Correct Answer: B**

Explanation: Each `f(&x;)` increments `x` and returns 1 (true). `&&` is left-to-right and all return true, so all three calls execute. `x` becomes 3. Condition is true. Output: All true: 3.

— End of Section 6: Decision Making —

12 Questions | Part of 165-Question Bank
© 2026 Tutorials Point India Private Limited

C Programming MCQ Question Bank

Section 7: Loops

12 Comprehensive Questions

Hackathon & Competitive Programming Level

With Detailed Explanations

Difficulty Levels

● Basic (4) ● Medium (4) ● Advanced (4)

Topics Covered

Loop Control Flow • break vs continue Behavior
Nested Loop Complexity • Infinite Loop Patterns
Loop Variable Scope • do-while Guarantees
Comma Operator in Loops • Post-increment in Conditions

Section 7: Loops

Q1. [Basic] What is the output of this program?

```
#include <stdio.h>

int main(void) {
    int i = 0;
    for (; i < 5; printf("%d ", i++))
        ;
    return 0;
}
```

- A) 0 1 2 3 4
- B) 1 2 3 4 5
- C) 0 1 2 3 4 5
- D) Nothing

✓ **Correct Answer: A**

Explanation: The for loop has empty init (i=0 before), condition i<5, and increment is printf("%d ", i++). The body is empty (just ;). Each iteration: check i<5, if true execute printf in increment part. Prints 0 1 2 3 4.

Q2. [Basic] What does this code print?

```
#include <stdio.h>

int main(void) {
    int i = 5;
    while (i-- > 0)
        printf("%d ", i);
    return 0;
}
```

- A) 5 4 3 2 1
- B) 4 3 2 1 0
- C) 5 4 3 2 1 0
- D) 4 3 2 1

✓ **Correct Answer: B**

Explanation: i-- > 0 is 'post-decrement greater than'. Check 5>0 (true), i becomes 4, print 4. Check 4>0, i becomes 3, print 3... Check 1>0, i becomes 0, print 0. Check 0>0 (false), stop. Output: 4 3 2 1 0.

Q3. [Basic] What is the output?

```
#include <stdio.h>

int main(void) {
    int i;
    for (i = 0; i < 5; i++) {
        if (i == 2)
            continue;
        if (i == 4)
            break;
        printf("%d ", i);
    }
    printf("| %d", i);
    return 0;
}
```

- A) 0 1 3 | 4
- B) 0 1 3 | 5
- C) 0 1 2 3 | 4
- D) 0 1 | 4

✓ **Correct Answer: A**

Explanation: i=0: print 0. i=1: print 1. i=2: continue skips print. i=3: print 3. i=4: break exits with i=4. Final printf shows i=4. Output: 0 1 3 | 4.

Q4. [Basic] How many times is 'X' printed?

```
#include <stdio.h>

int main(void) {
    int i = 10;
    do {
        printf("X");
        i++;
    } while (i < 5);
    return 0;
}
```

- A) 0
- B) 1
- C) 5
- D) Infinite

✓ **Correct Answer: B**

Explanation: do-while executes body at least once before checking condition. Body runs: print 'X', i becomes 11. Check 11 < 5 (false), exit. X printed exactly once.

Q5. [Medium] What does this code output?

```
#include <stdio.h>

int main(void) {
    int i, j;
    for (i = 0, j = 10; i < j; i++, j--)
        printf("%d ", i + j);
    return 0;
}
```

- A) 10 10 10 10 10
- B) 10 9 8 7 6
- C) 10 10 10
- D) 10 8 6 4 2

✓ **Correct Answer: A**

Explanation: i starts at 0, j at 10. Each iteration: i increases, j decreases, but sum i+j stays constant at 10. Loop runs while i < j: (0,10), (1,9), (2,8), (3,7), (4,6). When i=5, j=5, i<j false. Output: 10 10 10 10 10.

Q6. [Medium] What is the output?

```
#include <stdio.h>

int main(void) {
    int i = 0;
    while (i < 3) {
        int i = 10;
        printf("%d ", i);
        i++;
    }
    return 0;
}
```

- A) 10 11 12
- B) 10 10 10 ... (infinite loop)
- C) 0 1 2
- D) 10 10 10 (then stops)

✓ **Correct Answer: B**

Explanation: Inner 'int i = 10' shadows outer i. Inner i is reinitialized to 10 each iteration, incremented to 11, then destroyed. Outer i (checked in while) is never modified. Infinite loop printing 10 10 10...

Q7. [Medium] What does this print?

```
#include <stdio.h>

int main(void) {
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (j == 1)
                break;
            printf("%d%d ", i, j);
        }
    }
    return 0;
}
```

- A) 00 01 02 10 11 12 20 21 22
- B) 00 10 20
- C) 00
- D) 00 02 10 12 20 22

✓ **Correct Answer: B**

Explanation: Inner break only exits inner loop. For each i: j=0 prints, j=1 breaks inner loop. i=0: print 00, break. i=1: print 10, break. i=2: print 20, break. Output: 00 10 20.

Q8. [Medium] What is the value of sum?

```
#include <stdio.h>

int main(void) {
    int sum = 0;
    for (int i = 1; i <= 100; i++)
        for (int j = 1; j <= 100; j++)
            if (i == j)
                sum++;
    printf("%d", sum);
    return 0;
}
```

- A) 10000
- B) 100
- C) 200
- D) 5050

✓ **Correct Answer: B**

Explanation: sum++ only when i == j. In nested loops 1-100, i equals j exactly 100 times: (1,1), (2,2), ..., (100,100). Each happens once. sum = 100.

Q9. [Advanced] What is the output?

```
#include <stdio.h>

int main(void) {
    int i;
    for (i = 0; i++ < 5;)
        printf("%d ", i);
    printf("| %d", i);
    return 0;
}
```

- A) 1 2 3 4 5 | 6
- B) 0 1 2 3 4 | 5
- C) 1 2 3 4 | 5
- D) 1 2 3 4 5 | 5

✓ **Correct Answer: A**

Explanation: $i++ < 5$ is post-increment comparison. Compare with current value, then increment. $i=0$: $0 < 5$ true, i becomes 1, print 1. $i=1$: $1 < 5$ true, $i=2$, print 2... $i=4$: $4 < 5$ true, $i=5$, print 5. $i=5$: $5 < 5$ false, $i=6$, exit. Output: 1 2 3 4 5 | 6.

Q10. [Advanced] How many times does the inner loop body execute?

```
#include <stdio.h>

int main(void) {
    int count = 0;
    for (int i = 1; i <= 10; i++) {
        for (int j = 1; j <= 10; j += i) {
            count++;
        }
    }
    printf("%d", count);
    return 0;
}
```

- A) 100
- B) 55
- C) 30
- D) 33

✓ **Correct Answer: D**

Explanation: Inner loop: j goes 1, $1+i$, $1+2i$... up to 10. $i=1$: 10 iters. $i=2$: 5 iters (1,3,5,7,9). $i=3$: 4 iters (1,4,7,10). $i=4$: 3 (1,5,9). $i=5$: 2 (1,6). $i=6$: 2 (1,7). $i=7$: 2 (1,8). $i=8$: 2 (1,9). $i=9$: 2 (1,10). $i=10$: 1 (1). Total: $10+5+4+3+2+2+2+2+2+1 = 33$.

Q11. [Advanced] What is the output?

```

#include <stdio.h>

int main(void) {
    int x = 0;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (i == j)
                goto next;
            x++;
        }
        next:;
    }
    printf("%d", x);
    return 0;
}

```

- A) 0
- B) 3
- C) 6
- D) 9

✓ **Correct Answer: B**

Explanation: goto next jumps past inner loop to label. i=0,j=0: i==j, goto (x=0). i=1: j=0, x++ (x=1), j=1, i==j, goto. i=2: j=0, x++ (x=2), j=1, x++ (x=3), j=2, i==j, goto. Final x=3.

Q12. [Advanced] What does this print?

```

#include <stdio.h>

int main(void) {
    int a[3] = {1, 2, 3};
    int *p = a;
    int sum = 0;
    while (p < a + 3)
        sum += *p++;
    for (p = a + 2; p >= a; p--)
        sum += *p;
    printf("%d", sum);
    return 0;
}

```

- A) 6
- B) 12
- C) 9
- D) 18

✓ **Correct Answer: B**

Explanation: First while: sum += 1, 2, 3 = 6. p ends at a+3. Reset p = a+2 (pointing to last element). For loop backward: sum += a[2]=3, a[1]=2, a[0]=1 = 6 more. Total: 6 + 6 = 12.

— End of Section 7: Loops —

12 Questions | Part of 165-Question Bank
© 2026 Tutorials Point India Private Limited

C Programming MCQ Question Bank

Section 8: Functions

12 Comprehensive Questions

Hackathon & Competitive Programming Level

With Detailed Explanations

Difficulty Levels

● Basic (4) ● Medium (4) ● Advanced (4)

Topics Covered

Function Pointers • Callback Mechanisms • Recursion
Variadic Functions • Inline Functions • Parameter Passing
Return Value Semantics • Function Prototypes
Nested Function Calls • Stack Behavior

Section 8: Functions

Q1. [Basic] What is the output of this program?

```
#include <stdio.h>

void swap(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

int main(void) {
    int x = 10, y = 20;
    swap(x, y);
    printf("%d %d", x, y);
    return 0;
}
```

- A) 20 10
- B) 10 20
- C) 20 20
- D) 10 10

✓ **Correct Answer: B**

Explanation: C uses pass-by-value. The function receives copies of x and y. Swapping copies doesn't affect originals. x and y remain 10 and 20. To actually swap, use pointers: void swap(int *a, int *b).

Q2. [Basic] What does this code print?

```
#include <stdio.h>

int func(void) {
    static int count = 0;
    return ++count;
}

int main(void) {
    printf("%d ", func());
    printf("%d ", func());
    printf("%d", func());
    return 0;
}
```

- A) 1 1 1
- B) 1 2 3
- C) 0 1 2
- D) 3 3 3

✓ **Correct Answer: B**

Explanation: Static local variable 'count' is initialized once and persists across calls. First call: count becomes 1, return 1. Second: count becomes 2, return 2. Third: count becomes 3, return 3.

Q3. [Basic] What is the output?

```
#include <stdio.h>

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }

int main(void) {
    int (*fp)(int, int);
    fp = add;
    printf("%d ", fp(10, 5));
    fp = sub;
    printf("%d", fp(10, 5));
    return 0;
}
```

- A) 15 5
- B) 5 15
- C) 15 15
- D) Compilation error

✓ **Correct Answer: A**

Explanation: fp is a function pointer that can point to functions taking two ints and returning int. First fp=add, fp(10,5)=15. Then fp=sub, fp(10,5)=5. Output: 15 5.

Q4. [Basic] What happens with this code?

```
#include <stdio.h>

int main(void) {
    printf("%d", main);
    return 0;
}
```

- A) Prints 0
- B) Prints address of main function
- C) Compilation error
- D) Infinite recursion

✓ **Correct Answer: B**

Explanation: Function name without () gives its address. 'main' is the address of main function. %d prints it as integer (though %p would be proper). This compiles and prints the function's memory address.

Q5. [Medium] What is the output?

```
#include <stdio.h>

int f(int n) {
    if (n <= 1) return n;
    return f(n - 1) + f(n - 2);
}

int main(void) {
    printf("%d", f(6));
    return 0;
}
```

- A) 6
- B) 8
- C) 13
- D) 21

✓ **Correct Answer: B**

Explanation: This is Fibonacci. $f(6) = f(5) + f(4) = (f(4)+f(3)) + (f(3)+f(2)) = \dots$ Base: $f(0)=0$, $f(1)=1$. Sequence: 0,1,1,2,3,5,8. $f(6) = 8$.

Q6. [Medium] What does this print?

```
#include <stdio.h>

void modify(int arr[]) {
    arr[0] = 100;
    arr = NULL;
}

int main(void) {
    int a[] = {1, 2, 3};
    modify(a);
    printf("%d %d", a[0], a[1]);
    return 0;
}
```

- A) 1 2
- B) 100 2
- C) Crash
- D) 100 100

✓ **Correct Answer: B**

Explanation: Arrays decay to pointers when passed. $arr[0]=100$ modifies original array. But $arr=NULL$ only changes the local pointer copy, not the original array address. $a[0]$ is now 100, $a[1]$ remains 2.

Q7. [Medium] What is the output?

```
#include <stdio.h>
#include <stdarg.h>

int sum(int count, ...) {
    va_list args;
    va_start(args, count);
    int total = 0;
    for (int i = 0; i < count; i++)
        total += va_arg(args, int);
    va_end(args);
    return total;
}

int main(void) {
    printf("%d", sum(4, 10, 20, 30, 40));
    return 0;
}
```

- A) 4
- B) 100
- C) 40
- D) 10

✓ Correct Answer: B

Explanation: Variadic function with count=4 arguments. va_arg extracts each int: 10+20+30+40 = 100. The first parameter 'count' tells how many variadic arguments follow.

Q8. [Medium] What is the result?

```
#include <stdio.h>

int g(int);

int f(int n) {
    if (n <= 1) return 1;
    return g(n - 1);
}

int g(int n) {
    if (n <= 1) return 1;
    return n * f(n);
}

int main(void) {
    printf("%d", f(5));
    return 0;
}
```

- A) 120
- B) 24
- C) 5
- D) 4

✓ **Correct Answer: B**

Explanation: Mutual recursion.

$f(5) \rightarrow g(4) \rightarrow 4 * f(4) \rightarrow 4 * g(3) \rightarrow 4 * 3 * f(3) \rightarrow 4 * 3 * g(2) \rightarrow 4 * 3 * 2 * f(2) \rightarrow 4 * 3 * 2 * g(1) \rightarrow 4 * 3 * 2 * 1 = 24$. Pattern computes $(n-1)!$ when starting with $f(n)$.

Q9. [Advanced] What is the output?

```
#include <stdio.h>

int (*getFunc(int x))(int, int) {
    static int add(int a, int b) { return a + b; }
    static int mul(int a, int b) { return a * b; }
    return x ? add : mul;
}

int main(void) {
    printf("%d %d", getFunc(1)(3, 4), getFunc(0)(3, 4));
    return 0;
}
```

- A) 7 12
- B) 12 7
- C) Compilation error - nested functions
- D) 7 7

✓ **Correct Answer: C**

Explanation: Standard C does not support nested functions. This code uses a GCC extension. In strict C99/C11, this is a compilation error. Functions must be defined at file scope.

Q10. [Advanced] What does this print?

```
#include <stdio.h>

void f(int n) {
    if (n > 0) {
        f(n - 1);
        printf("%d ", n);
        f(n - 1);
    }
}

int main(void) {
    f(3);
    return 0;
}
```

- A) 1 2 1 3 1 2 1
- B) 3 2 1 1 2 1
- C) 1 2 3 2 1
- D) 1 1 2 1 1 3 1 1 2 1 1

✓ **Correct Answer: A**

Explanation: Double recursion: f(n-1), print n, f(n-1). f(3): f(2), print 3, f(2). f(2): f(1), print 2, f(1). f(1): f(0), print 1, f(0). Pattern creates: 1 2 1 3 1 2 1 (like binary tree inorder on complete tree).

Q11. [Advanced] What is the output?

```
#include <stdio.h>

int arr[] = {1, 2, 3, 4, 5};

int* f(int i) {
    return &arr[i];
}

int main(void) {
    *f(2) = 10;
    printf("%d %d %d", arr[1], arr[2], arr[3]);
    return 0;
}
```

- A) 2 10 4
- B) 2 3 4
- C) 10 10 10
- D) 1 10 3

✓ **Correct Answer: A**

Explanation: f(2) returns pointer to arr[2]. *f(2)=10 modifies arr[2] to 10. arr becomes {1,2,10,4,5}. Output: arr[1]=2, arr[2]=10, arr[3]=4.

Q12. [Advanced] What is the result?

```
#include <stdio.h>

int apply(int (*ops[])(int), int n, int val) {
    for (int i = 0; i < n; i++)
        val = ops[i](val);
    return val;
}

int dbl(int x) { return x * 2; }
int inc(int x) { return x + 1; }
int sqr(int x) { return x * x; }

int main(void) {
    int (*funcs[])(int) = {inc, dbl, sqr};
    printf("%d", apply(funcs, 3, 2));
    return 0;
}
```

- A) 36
- B) 25
- C) 18
- D) 10

✓ **Correct Answer: A**

Explanation: Array of function pointers applied in sequence. Start with 2. $\text{inc}(2)=3$. $\text{dbl}(3)=6$. $\text{sqr}(6)=36$. Functions compose: $\text{sqr}(\text{dbl}(\text{inc}(2))) = \text{sqr}(\text{dbl}(3)) = \text{sqr}(6) = 36$.

— End of Section 8: Functions —

C Programming MCQ Question Bank

Section 9: Scope Rules

12 Comprehensive Questions

Hackathon & Competitive Programming Level

With Detailed Explanations

Difficulty Levels

● Basic (4) ● Medium (4) ● Advanced (4)

Topics Covered

Block Scope • File Scope • Function Scope
Linkage (External, Internal, None) • Storage Duration
Name Hiding & Shadowing • extern Keyword
static at File Scope • Scope vs Lifetime

Section 9: Scope Rules

Q1. [Basic] What is the output of this program?

```
#include <stdio.h>

int x = 10;

void func(void) {
    int x = 20;
    {
        int x = 30;
        printf("%d ", x);
    }
    printf("%d ", x);
}

int main(void) {
    func();
    printf("%d", x);
    return 0;
}
```

- A) 30 20 10
- B) 10 20 30
- C) 30 30 30
- D) 10 10 10

✓ **Correct Answer: A**

Explanation: Each block creates a new scope. Innermost x=30 shadows x=20, which shadows global x=10. Inner block prints 30, after block ends x=20 is visible (prints 20), main sees global x=10.

Q2. [Basic] What does this code print?

```

#include <stdio.h>

int a = 5;

void f(void) {
    int a = 10;
    printf("%d ", a);
    a++;
}

int main(void) {
    f();
    f();
    printf("%d", a);
    return 0;
}

```

- A) 10 10 5
- B) 10 11 5
- C) 10 10 12
- D) 5 5 5

✓ **Correct Answer: A**

Explanation: Local 'a' in f() shadows global 'a'. Each call creates new local a=10, prints 10, increments to 11, then destroyed. Global a=5 is never modified. Output: 10 10 5.

Q3. [Basic] What is the output?

```

#include <stdio.h>

void counter(void) {
    static int count = 0;
    count++;
    printf("%d ", count);
}

int main(void) {
    counter();
    counter();
    counter();
    return 0;
}

```

- A) 1 1 1
- B) 1 2 3
- C) 0 1 2
- D) 3 3 3

✓ **Correct Answer: B**

Explanation: Static local variable has block scope but static storage duration. Initialized once, persists between calls. count goes 0->1->2->3 across calls. Output: 1 2 3.

Q4. [Basic] Which statement about this code is correct?

```
// file1.c
int global = 100;

// file2.c
extern int global;

void print_global(void) {
    printf("%d", global);
}
```

- A) Compilation error - global redefined
- B) extern declares global, linking finds definition in file1.c
- C) Each file has its own copy of global
- D) extern creates a new variable

✓ **Correct Answer: B**

Explanation: 'extern int global' is a declaration (not definition). It tells compiler that 'global' exists elsewhere with external linkage. Linker resolves it to the definition in file1.c. Both files share the same variable.

Q5. [Medium] What is the output?

```
#include <stdio.h>

static int x = 10;

void f(void) {
    static int x = 20;
    x++;
    printf("%d ", x);
}

int main(void) {
    f();
    x++;
    f();
    printf("%d", x);
    return 0;
}
```

- A) 21 22 11
- B) 21 11 22
- C) 21 22 12
- D) 11 12 21

✓ **Correct Answer: A**

Explanation: Two different static x: file-scope x=10 and function-scope x=20. f() uses its own static x: first call 20->21, print 21. main increments file x: 10->11. Second f() call: 21->22, print 22. Final file x=11.

Q6. [Medium] What does this print?

```
#include <stdio.h>

int x = 1;

int f(void) {
    int x = 2;
    {
        extern int x;
        return x;
    }
}

int main(void) {
    printf("%d", f());
    return 0;
}
```

- A) 1
- B) 2
- C) 0
- D) Undefined behavior

✓ **Correct Answer: A**

Explanation: 'extern int x' inside the block refers to the global x with external linkage, bypassing the local x=2. It returns global x=1. extern can access global even when shadowed by local variable.

Q7. [Medium] What is the result?

```
#include <stdio.h>

int arr[3] = {1, 2, 3};

void modify(void) {
    static int *p = arr;
    *p += 10;
    p++;
}

int main(void) {
    modify();
    modify();
    printf("%d %d %d", arr[0], arr[1], arr[2]);
    return 0;
}
```

- A) 11 12 3
- B) 11 2 3
- C) 1 12 3
- D) 11 12 13

✓ **Correct Answer: A**

Explanation: Static pointer `p` persists. First call: `*p (arr[0]) +=10` becomes 11, `p` moves to `arr[1]`. Second call: `*p (arr[1]) +=10` becomes 12, `p` moves to `arr[2]`. `arr = {11, 12, 3}`.

Q8. [Medium] What happens with this code?

```
#include <stdio.h>

int* danger(void) {
    int local = 42;
    return &local;
}

int main(void) {
    int *p = danger();
    printf("%d", *p);
    return 0;
}
```

- A) Prints 42
- B) Prints 0
- C) Undefined behavior - dangling pointer
- D) Compilation error

✓ **Correct Answer: C**

Explanation: 'local' has automatic storage duration, destroyed when `danger()` returns. Returning its address creates a dangling pointer. Accessing `*p` is undefined behavior - may print 42, garbage, or crash.

Q9. [Advanced] What is the output?

```
#include <stdio.h>

int x;

int main(void) {
    int x = 1;
    {
        int x = 2;
        {
            extern int x;
            printf("%d ", x);
        }
        printf("%d ", x);
    }
    printf("%d", x);
    return 0;
}
```

- A) 0 2 1
- B) 2 2 1
- C) 1 2 1
- D) 0 1 2

✓ Correct Answer: A

Explanation: Global x is implicitly 0 (uninitialized global). extern x refers to global (0). After that block, x=2 is visible. After outer block, x=1 is visible. Output: 0 2 1.

Q10. [Advanced] Which statement is TRUE about this code?

```
// file1.c
static int count = 0;
void increment(void) { count++; }
int get_count(void) { return count; }

// file2.c
static int count = 100;
void decrement(void) { count--; }
int get_count2(void) { return count; }
```

- A) Linker error - count defined twice
- B) Both files share the same count variable
- C) Each file has its own private count (internal linkage)
- D) Compilation error

✓ Correct Answer: C

Explanation: 'static' at file scope gives internal linkage. Each translation unit has its own private 'count'. No conflict because they're invisible outside their files. file1's count starts at 0, file2's at 100.

Q11. [Advanced] What is the output?

```
#include <stdio.h>

void outer(void) {
    int x = 10;

    void inner(void) {
        printf("%d", x);
    }

    inner();
}

int main(void) {
    outer();
    return 0;
}
```

- A) 10
- B) 0
- C) Compilation error - nested functions not allowed
- D) Undefined behavior

✓ **Correct Answer: C**

Explanation: Standard C does not support nested function definitions. Functions must be defined at file scope. This is a GCC extension. Strictly conforming C code would not compile this.

Q12. [Advanced] What is the result?

```

#include <stdio.h>

int f(void);

int x = 10;
static int y = 20;

int f(void) {
    static int x = 30;
    int y = 40;
    return x++ + y++;
}

int main(void) {
    printf("%d ", f());
    printf("%d ", f());
    printf("%d %d", x, y);
    return 0;
}

```

- A) 70 71 10 20
- B) 70 72 10 20
- C) 70 71 11 21
- D) 70 70 10 20

✓ **Correct Answer: A**

Explanation: f() has static x=30 and local y=40. First call: 30+40=70, static x becomes 31, local y destroyed. Second call: 31+40=71, static x becomes 32. Global x=10 and file-static y=20 unchanged. Output: 70 71 10 20.

— End of Section 9: Scope Rules —

12 Questions | Part of 165-Question Bank
 © 2026 Tutorials Point India Private Limited

C Programming MCQ Question Bank

Section 10: Files and I/O

15 Comprehensive Questions

Hackathon & Competitive Programming Level

With Detailed Explanations

Difficulty Levels

● Basic (5) ● Medium (5) ● Advanced (5)

Topics Covered

File Opening Modes • fopen/fclose • fread/fwrite
fprintf/fscanf • fgets/fputs • fseek/ftell/rewind
Binary vs Text Mode • EOF Detection • Error Handling
stdin/stdout/stderr • Buffering • File Positioning

Section 10: Files and I/O

Q1. [Basic] What does fopen() return if the file cannot be opened?

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("nonexistent.txt", "r");
    if (fp == /* ? */)
        printf("Failed");
    return 0;
}
```

- A) -1
- B) 0
- C) NULL
- D) EOF

✓ **Correct Answer: C**

Explanation: fopen() returns NULL on failure (file doesn't exist, permission denied, etc.). Always check for NULL before using the file pointer. NULL is a null pointer constant, typically defined as ((void*)0).

Q2. [Basic] What is the difference between 'w' and 'a' mode in fopen()?

```
FILE *f1 = fopen("file.txt", "w");
FILE *f2 = fopen("file.txt", "a");
```

- A) 'w' truncates file, 'a' appends to end
- B) Both create new file if not exists
- C) 'a' truncates file, 'w' appends
- D) No difference

✓ **Correct Answer: A**

Explanation: 'w' (write) truncates existing file to zero length or creates new. 'a' (append) opens for writing at end, preserving existing content. Both create file if it doesn't exist.

Q3. [Basic] What does this code print?

```
#include <stdio.h>

int main(void) {
    printf("Hello");
    fprintf(stderr, "Error");
    printf("World");
    return 0;
}
```

- A) HelloErrorWorld
- B) HelloWorldError
- C) ErrorHelloWorld
- D) Output order may vary due to buffering

✓ **Correct Answer: D**

Explanation: stdout is typically line-buffered, stderr is unbuffered. 'Error' may appear before 'HelloWorld' because stderr flushes immediately while stdout buffers. Actual order depends on buffering behavior.

Q4. [Basic] What is the return value of fgetc() at end of file?

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("test.txt", "r");
    int ch;
    while ((ch = fgetc(fp)) != /* ? */)
        putchar(ch);
    fclose(fp);
    return 0;
}
```

- A) NULL
- B) 0
- C) -1
- D) EOF

✓ **Correct Answer: D**

Explanation: fgetc() returns EOF (typically -1) on end-of-file or error. Note that fgetc() returns int, not char, because EOF must be distinguishable from valid char values (0-255).

Q5. [Basic] What does fgets() include in the string it reads?

```
char buf[100];
fgets(buf, 100, fp);
```

- A) Newline character if present, plus null terminator

- B) Only the text, no newline
- C) Text without null terminator
- D) Newline but no null terminator

✓ **Correct Answer: A**

Explanation: fgets() reads up to n-1 chars, stops at newline or EOF. It INCLUDES the newline if read, and ALWAYS adds null terminator. Compare to gets() which doesn't include newline (and is unsafe).

Q6. [Medium] What is the output of this program?

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("test.txt", "w+");
    fprintf(fp, "ABCDE");
    fseek(fp, 2, SEEK_SET);
    fprintf(fp, "XY");
    rewind(fp);
    char buf[10];
    fgets(buf, 10, fp);
    printf("%s", buf);
    fclose(fp);
    return 0;
}
```

- A) ABCDE
- B) ABXYE
- C) XYCDE
- D) ABXYDE

✓ **Correct Answer: B**

Explanation: Write 'ABCDE'. fseek to position 2. Write 'XY' overwrites positions 2-3. File becomes 'ABXYE'. rewind goes to start. fgets reads 'ABXYE'. Output: ABXYE.

Q7. [Medium] What does ftell() return?

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("test.txt", "w");
    fprintf(fp, "Hello World");
    long pos = ftell(fp);
    printf("%ld", pos);
    fclose(fp);
    return 0;
}
```

- A) 0

- B) 11
- C) 12
- D) -1

✓ **Correct Answer: B**

Explanation: `ftell()` returns current file position. After writing 'Hello World' (11 characters), position is 11 (0-indexed, so next write would be at position 11). `ftell()` returns -1L on error.

Q8. [Medium] What is the issue with this code?

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("data.bin", "rb");
    int arr[5];
    fread(arr, sizeof(int), 5, fp);
    // Use arr...
    fclose(fp);
    return 0;
}
```

- A) `fread` cannot read integers
- B) No check if `fopen` succeeded
- C) Should use 'r' not 'rb'
- D) `fread` returns void

✓ **Correct Answer: B**

Explanation: If `fopen` fails (file doesn't exist), `fp` is `NULL`. Using `NULL` with `fread` causes undefined behavior (likely crash). Always check: `if (fp == NULL) { handle error }`. Also should check `fread`'s return value.

Q9. [Medium] What does this code output?

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("test.txt", "w");
    fputs("Line1\nLine2", fp);
    fclose(fp);

    fp = fopen("test.txt", "r");
    char buf[20];
    fgets(buf, 20, fp);
    printf("[%s]", buf);
    fclose(fp);
    return 0;
}
```

- A) `[Line1\nLine2]`

- B) [Line1]
- C) [Line1\n]
- D) [Line1]

✓ **Correct Answer: C**

Explanation: fputs writes 'Line1\nLine2'. fgets reads until newline or EOF. It reads 'Line1\n' and stops (newline included). buf contains 'Line1\n\0'. Output shows [Line1 followed by newline then].

Q10. [Medium] What is the correct way to check for read errors vs EOF?

```
FILE *fp = fopen("file.txt", "r");
int ch;
while ((ch = fgetc(fp)) != EOF) {
    // process ch
}
// After loop, how to distinguish error from EOF?
```

- A) Check if ch == -1 for error
- B) Use feof(fp) for EOF, ferror(fp) for error
- C) EOF always means error
- D) Cannot distinguish them

✓ **Correct Answer: B**

Explanation: After loop exits with EOF, call feof(fp) to check if end-of-file was reached, and ferror(fp) to check if an error occurred. Both conditions return EOF from fgetc(), so these functions distinguish them.

Q11. [Advanced] What is the output assuming test.txt contains 'ABCDEF'?

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("test.txt", "r+");
    fseek(fp, 0, SEEK_END);
    long size = ftell(fp);
    fseek(fp, -2, SEEK_END);
    char buf[3];
    fread(buf, 1, 2, fp);
    buf[2] = '\0';
    printf("%s %ld", buf, size);
    fclose(fp);
    return 0;
}
```

- A) EF 6
- B) DE 6
- C) CD 6
- D) EF 7

✓ **Correct Answer: A**

Explanation: SEEK_END goes to end (position 6). ftell returns 6. fseek -2 from end goes to position 4. fread reads 2 bytes: 'E' and 'F'. Output: EF 6.

Q12. [Advanced] What happens with this binary file operation?

```
#include <stdio.h>

struct Data {
    int id;
    char name[20];
};

int main(void) {
    struct Data d1 = {1, "Alice"};
    FILE *fp = fopen("data.bin", "wb");
    fwrite(&d1, sizeof(struct Data), 1, fp);
    fclose(fp);

    struct Data d2;
    fp = fopen("data.bin", "rb");
    size_t n = fread(&d2, sizeof(struct Data), 1, fp);
    printf("%d %s %zu", d2.id, d2.name, n);
    fclose(fp);
    return 0;
}
```

- A) 1 Alice 1
- B) 1 Alice 24
- C) Garbage values
- D) Platform-dependent due to struct padding

✓ **Correct Answer: A**

Explanation: fwrite writes entire struct as binary. fread reads it back. fread returns number of items read (1). Works correctly on same platform. However, binary files may not be portable across different architectures due to endianness and padding.

Q13. [Advanced] What is the output?

```
#include <stdio.h>

int main(void) {
    FILE *fp = tmpfile();
    if (fp) {
        fprintf(fp, "Temporary");
        rewind(fp);
        char buf[20];
        fgets(buf, 20, fp);
        printf("%s", buf);
        // Note: no fclose
    }
    return 0;
}
```

- A) Temporary (file auto-deleted on close/exit)
- B) Nothing - tmpfile fails
- C) Memory leak
- D) Compilation error

✓ **Correct Answer: A**

Explanation: tmpfile() creates a temporary file that is automatically deleted when closed or program exits. Even without explicit fclose, the file is removed on program termination. Output: Temporary.

Q14. [Advanced] What does setvbuf do in this code?

```
#include <stdio.h>

int main(void) {
    char buffer[1024];
    setvbuf(stdout, buffer, _IOFBF, 1024);
    printf("Hello");
    printf("World");
    // No newline, no fflush
    return 0;
}
```

- A) Prints HelloWorld immediately
- B) Prints nothing until buffer full or fflush
- C) Prints HelloWorld at program exit
- D) Compilation error

✓ **Correct Answer: C**

Explanation: _IOFBF sets full buffering (1024 bytes). Output is buffered until: buffer fills, fflush called, or program exits normally. 'HelloWorld' (10 bytes) doesn't fill buffer, so it prints at exit when buffers are flushed.

Q15. [Advanced] What is the issue with this scanf usage?

```
#include <stdio.h>

int main(void) {
    int x;
    char str[100];
    printf("Enter number: ");
    scanf("%d", &x);
    printf("Enter string: ");
    fgets(str, 100, stdin);
    printf("x=%d str=[%s]", x, str);
    return 0;
}
```

- A) Works correctly
- B) fgets reads leftover newline from scanf
- C) scanf and fgets cannot be mixed
- D) Buffer overflow

✓ **Correct Answer: B**

Explanation: scanf('%d') leaves '\n' in input buffer after reading number. fgets immediately reads this leftover newline as an empty line. Fix: add getchar() after scanf, or use scanf(' %m[^\\n]') or fgets for both inputs.

— End of Section 10: Files and I/O —

15 Questions | Part of 165-Question Bank
© 2026 Tutorials Point India Private Limited

C Programming MCQ Question Bank

Section 11: Arrays

12 Comprehensive Questions

Hackathon & Competitive Programming Level

With Detailed Explanations

Difficulty Levels

● Basic (4) ● Medium (4) ● Advanced (4)

Topics Covered

Array Decay to Pointer • Multi-dimensional Arrays
Array Initialization • sizeof with Arrays • VLAs
Array Parameter Passing • Pointer Arithmetic
Row-major Order • Array of Pointers vs 2D Array

Section 11: Arrays

Q1. [Basic] What is the output of this program?

```
#include <stdio.h>

int main(void) {
    int arr[5] = {1, 2, 3};
    printf("%d %d %d", arr[2], arr[3], arr[4]);
    return 0;
}
```

- A) 3 0 0
- B) 3 garbage garbage
- C) 1 2 3
- D) Compilation error

✓ **Correct Answer: A**

Explanation: Partial initialization: arr[0]=1, arr[1]=2, arr[2]=3. Remaining elements arr[3] and arr[4] are implicitly initialized to 0. This is guaranteed by the C standard for partial initialization.

Q2. [Basic] What does this code print?

```
#include <stdio.h>

int main(void) {
    int arr[] = {10, 20, 30, 40, 50};
    int *p = arr;
    printf("%d %d", *(p + 2), p[3]);
    return 0;
}
```

- A) 30 40
- B) 20 30
- C) 30 50
- D) 20 40

✓ **Correct Answer: A**

Explanation: p points to arr[0]. *(p + 2) is arr[2] = 30. p[3] is equivalent to *(p + 3) = arr[3] = 40. Array subscript and pointer arithmetic are interchangeable.

Q3. [Basic] What is the output?

```
#include <stdio.h>

int main(void) {
    int arr[3] = {1, 2, 3};
    printf("%zu %zu", sizeof(arr), sizeof(arr[0]));
    return 0;
}
```

- A) 12 4
- B) 3 4
- C) 12 1
- D) 3 1

✓ **Correct Answer: A**

Explanation: `sizeof(arr)` returns total bytes: $3 * \text{sizeof(int)} = 3 * 4 = 12$ bytes. `sizeof(arr[0])` returns size of one element: `sizeof(int) = 4` bytes. Number of elements = `sizeof(arr)/sizeof(arr[0]) = 3`.

Q4. [Basic] What is the result?

```
#include <stdio.h>

void func(int arr[]) {
    printf("%zu", sizeof(arr));
}

int main(void) {
    int arr[10] = {0};
    func(arr);
    return 0;
}
```

- A) 40
- B) 10
- C) 8 (or 4 on 32-bit)
- D) 1

✓ **Correct Answer: C**

Explanation: When array is passed to function, it decays to pointer. 'int arr[]' parameter is actually 'int *arr'. `sizeof(arr)` inside func returns pointer size (8 on 64-bit, 4 on 32-bit), not array size.

Q5. [Medium] What is the output?

```
#include <stdio.h>

int main(void) {
    int arr[2][3] = {{1, 2, 3}, {4, 5, 6}};
    printf("%d %d", arr[1][2], (*(arr + 1) + 2));
    return 0;
}
```

- A) 6 6
- B) 5 6
- C) 6 5
- D) 3 6

✓ **Correct Answer: A**

Explanation: `arr[1][2] = 6` (row 1, col 2). `*(arr + 1) + 2`: `arr+1` points to row 1, `*(arr+1)` is row 1 array, `+2` moves to element 2, dereference gives 6. Both expressions are equivalent.

Q6. [Medium] What does this print?

```
#include <stdio.h>

int main(void) {
    int arr[3][3] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
    int *p = &arr[0][0];
    printf("%d %d", *(p + 4), arr[1][1]);
    return 0;
}
```

- A) 5 5
- B) 4 5
- C) 5 4
- D) 4 4

✓ **Correct Answer: A**

Explanation: 2D array is stored in row-major order: 1,2,3,4,5,6,7,8,9 contiguously. `p` points to `arr[0][0]`. `p+4` points to 5th element (index 4) = 5. `arr[1][1] = row1, col1 = element at index 4 = 5`.

Q7. [Medium] What is the output?

```
#include <stdio.h>

int main(void) {
    char *arr[] = {"Sun", "Mon", "Tue"};
    char **p = arr;
    printf("%c %c %s", **p, *(p + 1), *(p + 2));
    return 0;
}
```

- A) S M Tue
- B) Sun Mon Tue
- C) S M T
- D) Compilation error

✓ **Correct Answer: A**

Explanation: arr is array of char pointers. p points to arr[0]. **p = first char of arr[0] = 'S'. *(p+1) = first char of arr[1] = 'M'. *(p+2) = arr[2] = 'Tue' string. Output: S M Tue.

Q8. [Medium] What is the result?

```
#include <stdio.h>

int main(void) {
    int arr[5] = {1, 2, 3, 4, 5};
    int *p = arr + 2;
    printf("%d %d %d", p[-1], p[0], p[1]);
    return 0;
}
```

- A) 1 2 3
- B) 2 3 4
- C) 3 4 5
- D) Undefined behavior

✓ **Correct Answer: B**

Explanation: p points to arr[2] (value 3). p[-1] = *(p-1) = arr[1] = 2. p[0] = arr[2] = 3. p[1] = arr[3] = 4. Negative indices are valid as long as they stay within array bounds.

Q9. [Advanced] What is the output?


```
#include <stdio.h>

int main(void) {
    int arr[3][4];
    printf("%d", (int)(sizeof(arr) / sizeof(arr[0][0])));
    printf(" %d", (int)(sizeof(arr[0]) / sizeof(arr[0][0])));
    printf(" %d", (int)(sizeof(arr) / sizeof(arr[0])));
    return 0;
}
```

- A) 12 4 3
- B) 12 3 4
- C) 3 4 12
- D) 4 3 12

✓ **Correct Answer: A**

Explanation: Total elements: $3 \times 4 = 12$. $\text{sizeof(arr[0])} / \text{sizeof(arr[0][0])} = \text{columns} = 4$. $\text{sizeof(arr)} / \text{sizeof(arr[0])} = \text{rows} = 3$. This technique works to get dimensions of 2D arrays.

Q10. [Advanced] What does this code print?

```
#include <stdio.h>

int main(void) {
    int arr[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int (*p)[3] = arr;
    p++;
    printf("%d %d", (*p)[0], (*p)[1]);
    return 0;
}
```

- A) 1 2
- B) 4 5
- C) 2 3
- D) 3 4

✓ **Correct Answer: B**

Explanation: p is pointer to array of 3 ints. Initially points to arr[0] (first row). p++ advances by $\text{sizeof(int[3])} = 12$ bytes, now points to arr[1]. $(*p)[0] = \text{arr}[1][0] = 4$. $(*p)[1] = \text{arr}[1][1] = 5$.

Q11. [Advanced] What is the output?

```
#include <stdio.h>

int main(void) {
    int arr[] = {1, 2, 3, 4, 5};
    int *p = arr;
    int *q = &arr[4];
    printf("%td", q - p);
    return 0;
}
```

- A) 16
- B) 4
- C) 5
- D) 20

✓ **Correct Answer: B**

Explanation: Pointer subtraction gives number of elements between them, not bytes. q points to arr[4], p to arr[0]. Difference = 4 elements. %td is for ptrdiff_t. In bytes it would be 16 (4*sizeof(int)).

Q12. [Advanced] What is the result of this code?

```
#include <stdio.h>

void process(int n, int arr[n][n]) {
    int sum = 0;
    for (int i = 0; i < n; i++)
        sum += arr[i][i];
    printf("%d", sum);
}

int main(void) {
    int mat[3][3] = {{1,2,3}, {4,5,6}, {7,8,9}};
    process(3, mat);
    return 0;
}
```

- A) 45
- B) 15
- C) 12
- D) Compilation error in C89

✓ **Correct Answer: B**

Explanation: VLA parameters (C99+). Function sums diagonal: arr[0][0] + arr[1][1] + arr[2][2] = 1 + 5 + 9 = 15. This syntax allows passing 2D arrays with runtime-determined dimensions. Would error in C89.

C Programming MCQ Question Bank

Section 12: Strings

12 Comprehensive Questions

Hackathon & Competitive Programming Level

With Detailed Explanations

Difficulty Levels

● Basic (4) ● Medium (4) ● Advanced (4)

Topics Covered

String Literals vs Character Arrays • Null Terminator
strlen/strcpy/strcat/strcmp • Buffer Overflow Risks
String Manipulation • strtok Behavior
sprintf/snprintf • String Pointers • strstr/strchr

Section 12: Strings

Q1. [Basic] What is the output of this program?

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char str[] = "Hello";
    printf("%zu %zu", strlen(str), sizeof(str));
    return 0;
}
```

- A) 5 5
- B) 5 6
- C) 6 6
- D) 6 5

✓ **Correct Answer: B**

Explanation: strlen returns length excluding null terminator = 5. sizeof returns total array size including null terminator = 6 bytes. 'Hello' has 5 chars + 1 null = 6 bytes in array.

Q2. [Basic] What does this code print?

```
#include <stdio.h>

int main(void) {
    char *s1 = "Hello";
    char s2[] = "Hello";
    printf("%d", s1 == s2);
    return 0;
}
```

- A) 1
- B) 0
- C) Compilation error
- D) Undefined behavior

✓ **Correct Answer: B**

Explanation: s1 points to string literal in read-only memory. s2 is a separate character array on stack. They are at different memory addresses, so pointer comparison returns 0 (false). Use strcmp for content comparison.

Q3. [Basic] What is the output?

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char str[10] = "ABC";
    strcat(str, "DEF");
    printf("%s %zu", str, strlen(str));
    return 0;
}
```

- A) ABCDEF 6
- B) ABC 3
- C) ABCDEF 7
- D) Buffer overflow

✓ **Correct Answer: A**

Explanation: strcat appends 'DEF' to 'ABC', result is 'ABCDEF'. strlen returns 6. Buffer is 10 bytes, we use 7 (6 chars + null), so no overflow. strcat finds null terminator and appends there.

Q4. [Basic] What is the result of strcmp?

```
#include <stdio.h>
#include <string.h>

int main(void) {
    int r1 = strcmp("apple", "banana");
    int r2 = strcmp("cat", "cat");
    int r3 = strcmp("dog", "cat");
    printf("%d %d %d", r1 < 0, r2 == 0, r3 > 0);
    return 0;
}
```

- A) 1 1 1
- B) 0 1 1
- C) 1 0 1
- D) 1 1 0

✓ **Correct Answer: A**

Explanation: strcmp returns negative if s1 < s2, zero if equal, positive if s1 > s2 (lexicographic). 'apple' < 'banana' (negative), 'cat' == 'cat' (zero), 'dog' > 'cat' (positive). All conditions true: 1 1 1.

Q5. [Medium] What is the output?

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char str[] = "Hello World";
    char *token = strtok(str, " ");
    printf("%s ", token);
    token = strtok(NULL, " ");
    printf("%s ", token);
    printf("[%s]", str);
    return 0;
}
```

- A) Hello World [Hello World]
- B) Hello World [Hello]
- C) Hello World [Hello\0World]
- D) Compilation error

✓ **Correct Answer: B**

Explanation: strtok modifies original string! It replaces delimiter with null. After tokenization, str becomes 'Hello\0World'. First token='Hello', second='World'. printf on str stops at first null, showing [Hello].

Q6. [Medium] What does this print?

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char dest[20];
    char *result = strcpy(dest, "Hello");
    strcat(result, " World");
    printf("%s %d", dest, result == dest);
    return 0;
}
```

- A) Hello 1
- B) Hello World 1
- C) Hello World 0
- D) Undefined behavior

✓ **Correct Answer: B**

Explanation: strcpy returns dest pointer. result == dest is true (1). strcat appends to result (which is dest). dest now contains 'Hello World'. Output: Hello World 1.

Q7. [Medium] What is the output?

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char str[] = "ABCDEF";
    char *p = strchr(str, 'D');
    if (p)
        printf("%s %td", p, p - str);
    return 0;
}
```

- A) DEF 3
- B) D 3
- C) ABCD 3
- D) DEF 4

✓ **Correct Answer: A**

Explanation: strchr finds first occurrence of 'D', returns pointer to it. p points to 'D' in str. Printing p as string gives 'DEF' (from D to end). p - str = index = 3. Output: DEF 3.

Q8. [Medium] What is the result?

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char s1[10] = "Hello";
    char s2[10] = "Hello";
    s1[0] = 'J';
    printf("%d %d", strcmp(s1, s2), strncmp(s1, s2, 0));
    return 0;
}
```

- A) positive 0
- B) negative 0
- C) 0 0
- D) positive positive

✓ **Correct Answer: A**

Explanation: s1 is 'Jello', s2 is 'Hello'. 'J' > 'H' in ASCII, so strcmp(s1, s2) returns positive. strncmp with n=0 compares 0 characters, always returns 0. Output: positive 0.

Q9. [Advanced] What is the output?


```
#include <stdio.h>
#include <string.h>

int main(void) {
    char str[10] = "ABCDEFGHI";
    memmove(str + 2, str, 5);
    printf("%s", str);
    return 0;
}
```

- A) ABABCDEHI
- B) ABCDEFGHI
- C) ABABCDEGHI
- D) ABABABAHI

✓ **Correct Answer: A**

Explanation: memmove safely handles overlapping regions. Copies 5 bytes from str (ABCDE) to str+2. Result: positions 0-1 stay (AB), positions 2-6 get ABCDE, positions 7-8 stay (HI). Final: ABABCDEHI.

Q10. [Advanced] What does this code print?

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char haystack[] = "abcabcabc";
    char *needle = "cab";
    char *p = strstr(haystack, needle);
    int count = 0;
    while (p) {
        count++;
        p = strstr(p + 1, needle);
    }
    printf("%d", count);
    return 0;
}
```

- A) 1
- B) 2
- C) 3
- D) 0

✓ **Correct Answer: B**

Explanation: strstr finds substring. 'cab' found at index 2 (ab[cab]cab) and index 5 (abcab[cab]c). Third search from index 6 finds nothing. Count = 2. Note: overlapping matches found by advancing p+1.

Q11. [Advanced] What is the output?

```
#include <stdio.h>

int main(void) {
    char buf[20];
    int n = snprintf(buf, 5, "Hello World");
    printf("%s %d", buf, n);
    return 0;
}
```

- A) Hell 11
- B) Hello 11
- C) Hell 4
- D) Hello World 11

✓ **Correct Answer: A**

Explanation: snprintf writes at most size-1 chars plus null. With size=5, writes 4 chars + null: 'Hell\0'. Returns what WOULD have been written without limit = 11. buf='Hell', n=11.

Q12. [Advanced] What is the result of this code?

```
#include <stdio.h>
#include <string.h>

size_t my_strlen(const char *s) {
    const char *p = s;
    while (*p) p++;
    return p - s;
}

int main(void) {
    char str[] = "AB\0CD";
    printf("%zu %zu", my_strlen(str), sizeof(str));
    return 0;
}
```

- A) 5 6
- B) 2 6
- C) 2 5
- D) 5 5

✓ **Correct Answer: B**

Explanation: 'AB\0CD' has embedded null. my_strlen (like strlen) stops at first null, returns 2. But sizeof returns compile-time array size: 'A','B','\0','C','D','\0' = 6 bytes. Embedded nulls don't affect sizeof.

C Programming MCQ Question Bank

Section 13: Pointers

12 Comprehensive Questions

Hackathon & Competitive Programming Level

With Detailed Explanations

Difficulty Levels

● Basic (4) ● Medium (4) ● Advanced (4)

Topics Covered

Pointer Arithmetic • Double Pointers • void Pointers
Pointer to Functions • const with Pointers
Dynamic Memory • Pointer Aliasing • NULL Pointer
Pointer Casting • restrict Qualifier • Dangling Pointers

Section 13: Pointers

Q1. [Basic] What is the output of this program?

```
#include <stdio.h>

int main(void) {
    int a = 10, b = 20;
    int *p = &a, *q = &b;
    *p = *q;
    p = q;
    *p = 30;
    printf("%d %d", a, b);
    return 0;
}
```

- A) 20 30
- B) 10 30
- C) 30 30
- D) 20 20

✓ **Correct Answer: A**

Explanation: $*p = *q$ copies value: a becomes 20. $p = q$ makes p point to b. $*p = 30$ changes b to 30. a remains 20 (not affected by $p=q$). Final: a=20, b=30.

Q2. [Basic] What does this code print?

```
#include <stdio.h>

int main(void) {
    int arr[] = {10, 20, 30, 40};
    int *p = arr;
    printf("%d %d %d", *p, *(p+2), *p+2);
    return 0;
}
```

- A) 10 30 12
- B) 10 30 30
- C) 10 20 12
- D) 10 30 32

✓ **Correct Answer: A**

Explanation: $*p = arr[0] = 10$. $*(p+2) = arr[2] = 30$. $*p+2 = (*p)+2 = 10+2 = 12$ (dereference first, then add). Operator precedence: $*$ binds tighter than $+$.

Q3. [Basic] What is the output?

```
#include <stdio.h>

void modify(int **pp) {
    static int x = 100;
    *pp = &x;
}

int main(void) {
    int a = 10;
    int *p = &a;
    modify(&p);
    printf("%d", *p);
    return 0;
}
```

- A) 10
- B) 100
- C) Garbage
- D) Segmentation fault

✓ **Correct Answer: B**

Explanation: Double pointer pp allows modifying what p points to. *pp = &x makes p point to static x (value 100). Static ensures x exists after function returns. Output: 100.

Q4. [Basic] What is the result?

```
#include <stdio.h>

int main(void) {
    void *vp;
    int x = 10;
    float f = 3.14f;

    vp = &x;
    printf("%d ", *(int*)vp);
    vp = &f;
    printf("%.2f", *(float*)vp);
    return 0;
}
```

- A) 10 3.14
- B) Garbage Garbage
- C) Compilation error
- D) 10 0.00

✓ **Correct Answer: A**

Explanation: void* is generic pointer, can hold any address. Must cast before dereferencing. (int*)vp casts to int pointer, dereference gives 10. (float*)vp gives 3.14. void* provides type-agnostic storage.

Q5. [Medium] What is the output?

```
#include <stdio.h>

int main(void) {
    int a[] = {1, 2, 3, 4, 5};
    int *p = a;
    int *q = a + 4;

    while (p < q) {
        int temp = *p;
        *p++ = *q;
        *q-- = temp;
    }

    for (int i = 0; i < 5; i++)
        printf("%d ", a[i]);
    return 0;
}
```

- A) 5 4 3 2 1
- B) 1 2 3 4 5
- C) 5 2 3 4 1
- D) 1 4 3 2 5

✓ Correct Answer: A

Explanation: This reverses the array using two pointers. p starts at front, q at back. Swap and move towards center. Iteration: swap(1,5), swap(2,4). Middle 3 stays. Result: 5 4 3 2 1.

Q6. [Medium] What does this print?

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *p = malloc(3 * sizeof(int));
    p[0] = 10; p[1] = 20; p[2] = 30;

    int *q = realloc(p, 5 * sizeof(int));
    q[3] = 40; q[4] = 50;

    printf("%d %d", q[1], q[4]);
    printf(" %d", p == q);

    free(q);
    return 0;
}
```

- A) 20 50 and 0 or 1 (implementation-defined)
- B) 20 50 1
- C) Garbage 50 0
- D) Undefined behavior

✓ **Correct Answer: A**

Explanation: realloc may return same or different address. Original data preserved (q[1]=20). New elements added (q[4]=50). Whether p==q depends on whether realloc moved the block. Using p after realloc is risky if moved.

Q7. [Medium] What is the output?

```
#include <stdio.h>

int main(void) {
    const int x = 10;
    const int *p1 = &x;
    int * const p2 = (int*)&x;

    // *p1 = 20; // Line A
    // p1 = NULL; // Line B
    // *p2 = 20; // Line C
    // p2 = NULL; // Line D

    printf("%d %d", *p1, *p2);
    return 0;
}
```

- A) 10 10, Lines A and D would fail
- B) 10 10, Lines A and C would fail
- C) 10 10, Lines C and D would fail
- D) 10 10, Only Line A would fail

✓ **Correct Answer: A**

Explanation: const int *p1: pointer to const int - can't modify *p1 (A fails), can reassign p1 (B ok). int * const p2: const pointer to int - can modify *p2 (C ok syntax, but UB for const x), can't reassign p2 (D fails).

Q8. [Medium] What is the result?


```
#include <stdio.h>

int main(void) {
    char *p = "Hello";
    char *q = "Hello";
    char r[] = "Hello";
    char s[] = "Hello";

    printf("%d %d", p == q, r == s);
    return 0;
}
```

- A) 1 1
- B) 0 0
- C) 1 or 0 (impl-defined), 0
- D) 0, 1 or 0 (impl-defined)

✓ **Correct Answer: C**

Explanation: String literals may be merged by compiler (p==q could be true - same address). But arrays r and s are always separate stack objects, so r==s is always false (0). Literal merging is implementation-defined.

Q9. [Advanced] What is the output?

```
#include <stdio.h>

int main(void) {
    int a = 0x12345678;
    char *p = (char*)&a;

    printf("%02x %02x", p[0] & 0xff, p[3] & 0xff);
    return 0;
}
```

- A) 12 78 (big-endian) or 78 12 (little-endian)
- B) Always 12 78
- C) Always 78 12
- D) Undefined behavior

✓ **Correct Answer: A**

Explanation: Casting int* to char* allows byte-level access. Result depends on endianness. Little-endian (x86): LSB first, p[0]=0x78, p[3]=0x12. Big-endian: MSB first, p[0]=0x12, p[3]=0x78. System-dependent.

Q10. [Advanced] What does this code print?

```
#include <stdio.h>

int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

int main(void) {
    int (*ops[2])(int, int) = {add, mul};
    int (*(p))(int, int) = ops;

    printf("%d %d", (*p)(3, 4), (*(p+1))(3, 4));
    return 0;
}
```

- A) 7 12
- B) 12 7
- C) 3 4
- D) Compilation error

✓ **Correct Answer: A**

Explanation: ops is array of function pointers. p is pointer to function pointer, initialized to ops. *p = ops[0] = add. *(p+1) = ops[1] = mul. (*p)(3,4) = add(3,4) = 7. (*(p+1))(3,4) = mul(3,4) = 12.

Q11. [Advanced] What is the output?

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int **pp = malloc(2 * sizeof(int*));
    pp[0] = malloc(3 * sizeof(int));
    pp[1] = malloc(3 * sizeof(int));

    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 3; j++)
            pp[i][j] = i * 3 + j;

    printf("%d %d %d", pp[0][2], pp[1][0], *(*pp + 1));

    free(pp[0]); free(pp[1]); free(pp);
    return 0;
}
```

- A) 2 3 1
- B) 2 3 3
- C) 3 3 1
- D) 2 4 1

✓ **Correct Answer: A**

Explanation: 2D dynamic array: $pp[i][j] = i*3+j$. $pp[0][2] = 0*3+2 = 2$. $pp[1][0] = 1*3+0 = 3$. $*(pp+1) = *(pp[0]+1) = pp[0][1] = 0*3+1 = 1$. Output: 2 3 1.

Q12. [Advanced] What is the behavior of this code?

```
#include <stdio.h>
#include <stdint.h>

int main(void) {
    int x = 10;
    int *p = &x;

    uintptr_t addr = (uintptr_t)p;
    addr += sizeof(int);
    int *q = (int*)addr;

    printf("%p %p", (void*)p, (void*)q);
    return 0;
}
```

- A) Prints two addresses, q is sizeof(int) bytes after p
- B) Undefined behavior - invalid pointer arithmetic
- C) Compilation error
- D) Always segmentation fault

✓ **Correct Answer: A**

Explanation: `uintptr_t` allows integer arithmetic on addresses. Adding `sizeof(int)` to address is valid arithmetic. q points `sizeof(int)` bytes after x. Printing addresses is fine. Dereferencing q would be UB (no object there).

— End of Section 13: Pointers —

C Programming MCQ Question Bank

Section 14: User-Defined Types

6 Comprehensive Questions

Hackathon & Competitive Programming Level

With Detailed Explanations

Difficulty Levels

● Basic (2) ● Medium (2) ● Advanced (2)

Topics Covered

Structures • Unions • Bit Fields
typedef • Struct Padding & Alignment
Nested Structures • Anonymous Unions

Section 14: User-Defined Types

Q1. [Basic] What is the output of this program?

```
#include <stdio.h>

struct Point {
    int x;
    int y;
};

int main(void) {
    struct Point p1 = {10, 20};
    struct Point p2 = p1;
    p2.x = 30;
    printf("%d %d", p1.x, p2.x);
    return 0;
}
```

- A) 30 30
- B) 10 30
- C) 30 10
- D) 10 10

✓ **Correct Answer: B**

Explanation: Struct assignment creates a copy. `p2 = p1` copies all members. Modifying `p2.x` doesn't affect `p1`. `p1.x` remains 10, `p2.x` becomes 30. Structs are copied by value, not by reference.

Q2. [Basic] What does this code print?

```
#include <stdio.h>

union Data {
    int i;
    float f;
    char c;
};

int main(void) {
    union Data d;
    d.i = 65;
    printf("%d %c", d.i, d.c);
    return 0;
}
```

- A) 65 A
- B) 65 65
- C) 65 garbage

D) Undefined behavior

✓ **Correct Answer: A**

Explanation: Union members share same memory. d.i = 65 stores 65. On little-endian, LSB is 65 (0x41). d.c reads first byte = 65 = 'A' in ASCII. Both i and c interpret same memory differently.

Q3. [Medium] What is the likely output on a typical 64-bit system?

```
#include <stdio.h>

struct A {
    char c;
    int i;
    char d;
};

struct B {
    char c;
    char d;
    int i;
};

int main(void) {
    printf("%zu %zu", sizeof(struct A), sizeof(struct B));
    return 0;
}
```

- A) 6 6
- B) 12 8
- C) 9 6
- D) 12 12

✓ **Correct Answer: B**

Explanation: Struct padding for alignment. struct A: c(1) + pad(3) + i(4) + d(1) + pad(3) = 12. struct B: c(1) + d(1) + pad(2) + i(4) = 8. Reordering members can reduce size. int requires 4-byte alignment.

Q4. [Medium] What is the output?

```
#include <stdio.h>

struct Bits {
    unsigned int a : 3;
    unsigned int b : 5;
    unsigned int c : 8;
};

int main(void) {
    struct Bits bits = {7, 31, 255};
    bits.a++;
    printf("%u %u %u", bits.a, bits.b, bits.c);
    return 0;
}
```

- A) 8 31 255
- B) 0 31 255
- C) 7 31 255
- D) Overflow error

✓ **Correct Answer: B**

Explanation: Bit fields: a is 3 bits (max 7), b is 5 bits (max 31), c is 8 bits (max 255). a=7 (111 binary), a++ wraps to 0 (000) because 8 (1000) doesn't fit in 3 bits. Output: 0 31 255.

Q5. [Advanced] What is the output?

```
#include <stdio.h>

struct Node {
    int data;
    struct Node *next;
};

int main(void) {
    struct Node nodes[3] = {{1, &nodes[1]}, {2, &nodes[2]}, {3, NULL}};
    struct Node *p = &nodes[0];
    int sum = 0;

    while (p) {
        sum += p->data;
        p = p->next;
    }
    printf("%d", sum);
    return 0;
}
```

- A) 1
- B) 3
- C) 6

D) Compilation error

✓ **Correct Answer: C**

Explanation: Linked list in array. nodes[0]->nodes[1]->nodes[2]->NULL. Traverse: sum = 1 + 2 + 3 = 6. Self-referential struct with array initialization creates linked structure without malloc.

Q6. [Advanced] What is the result of this code?

```
#include <stdio.h>
#include <stddef.h>

struct Data {
    char a;
    int b;
    char c;
    double d;
};

int main(void) {
    printf("%zu %zu", offsetof(struct Data, b),
           offsetof(struct Data, d));
    return 0;
}
```

- A) 1 6
- B) 4 8
- C) 4 16
- D) 1 9

✓ **Correct Answer: C**

Explanation: offsetof gives byte offset from struct start. a at 0, b needs 4-byte alignment so at 4, c at 8, d needs 8-byte alignment so at 16. Layout: a(0) pad(1-3) b(4-7) c(8) pad(9-15) d(16-23). Offsets: b=4, d=16.

— End of Section 14: User-Defined Types —

★ **QUESTION BANK COMPLETE** ★

165 Total Questions Across 14 Sections
© 2026 Tutorials Point India Private Limited